

SECTION I

Dédicace

[Espace réservé à la dédicace personnelle]

Remerciements

[Espace réservé aux remerciements personnels]

Liste des figures

F1	Organigramme général d'Akieni	4
F2	Les concepts clés du langage UML	14
F3	Les types de diagrammes UML	15
F4	Structure en Y du processus 2TUP	17
F5	Diagramme de contexte statique de « SALISA »	22
F6	Diagramme de cas d'utilisation - P1 : Authentification	26
F7	Diagramme de cas d'utilisation - P2 : Profil prestataire	26
F8	Diagramme de cas d'utilisation - P3 : Gestion des missions	27
F9	Diagramme de cas d'utilisation - P4 : Recherche et matching	28
F10	Diagramme de cas d'utilisation - P5 : Messagerie et devis	28
F11	Diagramme de cas d'utilisation - P6 : Paiement	29
F12	Diagramme de cas d'utilisation - P7 : Notation et réputation	29
F13	Diagramme de cas d'utilisation - P8 : Notifications	30
F14	Diagramme de cas d'utilisation - P9 : Administration	30
F15	Diagramme de packages de « SALISA »	31
F16	Diagramme de séquence - inscription via numéro de téléphone (UC-01) . . .	35
F17	Diagramme de séquence - envoi de devis, paiement et validation de mission .	36
F18	Diagramme d'activités - publication de mission et sélection du prestataire . .	37
F19	Diagramme d'activités - processus de paiement mobile	38
F20	Diagramme de classes du système « SALISA »	40
F21	Diagramme d'objets - scénario de publication et réponse à une mission	41
F22	Diagramme de déploiement de « SALISA »	42
F23	Liste détaillée des tâches du projet « SALISA » sous MS Project	46
F24	Diagramme de Gantt du projet « SALISA »	47

Liste des tableaux

T1	Correspondance entre les phases 2TUP et les diagrammes UML utilisés	18
T2	Cas d'utilisation du package P1 : Authentification	23
T3	Cas d'utilisation du package P2 : Profil prestataire	23
T4	Cas d'utilisation du package P3 : Gestion des missions	23
T5	Cas d'utilisation du package P4 : Recherche et matching	24
T6	Cas d'utilisation du package P5 : Messagerie et devis	24
T7	Cas d'utilisation du package P6 : Paiement	24
T8	Cas d'utilisation du package P7 : Notation et réputation	24
T9	Cas d'utilisation du package P8 : Notifications	25
T10	Cas d'utilisation du package P9 : Administration	25
T11	Classification des principaux cas d'utilisation de « SALISA »	32
T12	Description textuelle de UC-01 : s'inscrire via numéro de téléphone	33
T13	Description textuelle de UC-11 : publier une mission	34
T14	Intervenants du projet « SALISA »	44
T15	Planification des tâches du projet « SALISA »	45
T16	Estimation des charges du projet « SALISA »	48

Abréviations et Sigles

Abrev. / Sigles	Signification
ESGAE	École Supérieure de Gestion et d'Administration des Entreprises
API	Application Programming Interface
FCFA	Franco de la Coopération Financière en Afrique centrale
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JWT	JSON Web Token
LP-GL	Licence Professionnelle - Génie Logiciel
MoMo	Mobile Money
MTN	Mobile Telephone Network
SMS	Short Message Service
OTP	One-Time Password
PWA	Progressive Web Application
REST	Representational State Transfer
BD	Base de Données
SGBD	Système de Gestion de Bases de Données
SQL	Structured Query Language
UML	Unified Modeling Language
UP	Unified Process (Processus Unifié)
2TUP	Two Tracks Unified Process
UC	Use Case (Cas d'utilisation)
RG	Règle de Gestion
URL	Uniform Resource Locator
WSS	WebSocket Secure

Sommaire

SECTION I

Dédicace	I
Remerciements	II
Liste des figures	III
Liste des tableaux	IV
Abréviations et sigles	V

SECTION II

Introduction	1
--------------------	---

Première Partie : Présentation Générale

Chapitre I : La structure d'accueil et le sujet	3
---	---

Deuxième Partie : Analyse et Conception

Chapitre I : Étude préliminaire et Méthode	10
Chapitre II : Analyse et Conception	19

Troisième Partie : Évaluation et Réalisation du Projet

Chapitre I : L'évaluation du projet	44
Chapitre II : Les outils et les techniques utilisés	49

Conclusion	53
------------------	----

SECTION III

Table des matières	a
Bibliographie	d
Annexes	e

SECTION II

Introduction

Le numérique a changé la façon dont les personnes accèdent aux services professionnels. Aujourd'hui, des plateformes numériques permettent de trouver facilement un professionnel compétent pour réaliser une tâche précise, de consulter son profil, de le contacter et de le payer en toute sécurité. Cependant, dans bien des environnements, cette mise en relation entre prestataires et clients reste informelle, peu structurée et difficile à organiser. C'est dans ce contexte qu'Akieni, société spécialisée dans la transformation digitale au Congo, a exprimé le besoin de disposer d'une solution numérique capable de connecter de manière fiable des prestataires de services professionnels avec des clients. C'est ainsi qu'est né « SALISA », notre solution pour répondre directement à ce besoin.

La problématique principale de ce travail peut être formulée ainsi : comment concevoir et réaliser une application cross-plateforme qui permet de connecter efficacement des prestataires et des clients, en garantissant la qualité des services, la sécurité des transactions et une expérience simple pour tous les utilisateurs ?

Pour répondre à cette problématique, nous formulons les hypothèses suivantes. Premièrement, un système de vérification d'identité et de notation permettra d'établir la confiance entre les utilisateurs de la plateforme. Deuxièmement, un algorithme de mise en correspondance basé sur les compétences, la localisation et le budget aidera les clients à trouver rapidement le bon prestataire. Troisièmement, l'intégration du paiement mobile via MTN MoMo et Airtel Money, avec un mécanisme de confirmation de transaction, protégera les deux parties lors des échanges. Quatrièmement, l'utilisation du processus 2TUP combiné au langage de modélisation UML permettra de concevoir un système solide et bien structuré.

Ce travail s'intitule : « **Conception et réalisation d'une application cross-plateforme de mise en relation entre prestataires et clients pour services professionnels à Akieni** ». Il est organisé en trois grandes parties. La première présente la structure d'accueil Akieni ainsi que le cadrage général du travail. La deuxième est consacrée à l'analyse des besoins et à la conception du système selon la démarche 2TUP et UML. La troisième décrit les outils utilisés, le planning et les résultats obtenus.

Première Partie

PRÉSENTATION GÉNÉRALE

Chapitre I : La structure d'accueil et le sujet

1.1 La structure d'accueil

1.1.1 Historique

Akieni est une société spécialisée dans la transformation numérique et les solutions logicielles en République du Congo. Implantée à Brazzaville depuis 2022, elle a pour mission d'accompagner les entreprises et les institutions publiques dans leurs projets de transformation digitale. Elle est reconnue comme un acteur majeur du numérique dans la sous-région de l'Afrique centrale.

Parmi ses initiatives, Akieni a lancé le programme de formation *Akieni Academy*, dont la première promotion a démarré officiellement le 15 novembre 2024 à Brazzaville. À cette occasion, 120 apprenants ont été sélectionnés parmi 1 740 candidats présélectionnés en ligne, venant du Congo, du Cameroun, du Rwanda et de la République Démocratique du Congo. La deuxième promotion a été lancée en janvier 2026, avec l'ambition de former 400 nouveaux apprenants. L'objectif à long terme d'Akieni est de former 30 000 jeunes aux métiers du numérique d'ici 2030, afin de positionner le Congo comme un pôle d'exportation de compétences digitales. La structure est dirigée par M. Frédéric NZÉ, actuel Ministre des Postes, des Télécommunications et de l'Économie numérique de la République du Congo.

1.1.2 Missions

Akieni a plusieurs missions principales :

- former des jeunes de 16 à 30 ans aux métiers du numérique, notamment en développement web Full Stack, en développement Python, en science des données et en conception UX/UI ;
- proposer des formations gratuites, d'une durée de six à neuf mois, basées sur des projets pratiques et concrets ;
- favoriser l'insertion professionnelle des apprenants en les connectant à des entreprises partenaires comme MTN Congo ;
- accompagner les organisations publiques et privées dans leurs projets de transformation digitale ;
- promouvoir les talents locaux et organiser des événements numériques tels que des hackathons pour encourager l'innovation.

1.1.3 Organigramme Général

La figure 1 présente l'organisation générale d'Akieni, structurée autour d'une Direction Générale qui supervise cinq directions principales, chacune pilotant ses propres départements opérationnels.

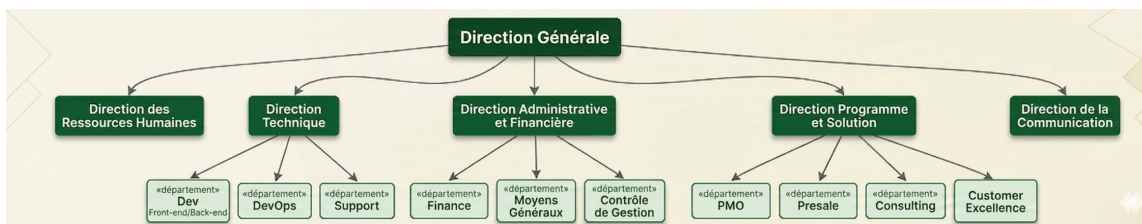


Figure 1 : Organigramme général d'Akieni

1.1.4 Attributions des structures

Akieni est organisée en cinq directions principales, chacune ayant un domaine de responsabilité bien défini.

La Direction des Ressources Humaines gère le recrutement, la gestion des carrières et le bien-être du personnel. Elle veille à ce que les équipes disposent des compétences nécessaires pour mener à bien les projets de la structure.

La Direction Technique est le cœur opérationnel d'Akieni sur le plan technologique. Elle regroupe trois départements : le département de développement Front-end et Back-end, qui conçoit et maintient les applications et plateformes numériques; le département DevOps, qui assure l'intégration continue, le déploiement et la supervision des infrastructures; et le département Support, qui gère l'assistance technique auprès des utilisateurs internes et des clients.

La Direction Administrative et Financière assure la gestion des ressources financières et administratives de la structure. Elle comprend le département Finance, chargé de la comptabilité et du suivi budgétaire; le département Moyens Généraux, qui gère les équipements et les locaux; et le département Contrôle de Gestion, qui évalue la performance financière de l'organisation.

La Direction Programme et Solution pilote les projets clients et les offres de services d'Akieni. Elle s'appuie sur quatre départements : le PMO (*Project Management Office*), qui coordonne et standardise la gestion des projets; le département Presale, qui accompagne les avant-ventes et la rédaction des offres commerciales; le département Consulting, qui propose des conseils en transformation numérique; et le département Customer Excellence, qui veille à la satisfaction client et à l'excellence de la qualité des livrables.

La Direction de la Communication est responsable de la communication interne et externe de la structure, de la gestion de l'image de marque d'Akieni, ainsi que de la promotion de ses programmes et de ses offres auprès du grand public et des partenaires.

1.1.5 Situation informatique

1.1.5.1 Personnel informatique

Akieni étant une structure spécialisée dans le numérique, l'essentiel de son personnel est à profil technique. L'équipe informatique est rattachée à la Direction Technique et se compose notamment de développeurs Full Stack (Front-end et Back-end), d'ingénieurs DevOps chargés de l'infrastructure et du déploiement, et de techniciens support. La Direction Programme et Solution compte également des consultants et des chefs de projet qui encadrent les missions techniques chez les clients.

1.1.5.2 Matériels informatiques

Pour mener ses activités de formation, de développement et de conseil, Akieni dispose d'un parc matériel adapté à ses besoins. On y trouve notamment des postes de travail et des ordinateurs portables mis à disposition des équipes techniques et des apprenants, des serveurs locaux utilisés pour les environnements de développement et de test, ainsi que les équipements réseau nécessaires à la connectivité de ses locaux (routeurs, switches, points d'accès Wi-Fi). Les projections et présentations en salle de formation sont assurées par des vidéoprojecteurs et des écrans interactifs.

1.1.5.3 Logiciels informatiques

Akieni utilise un ensemble d'outils logiciels couvrant ses différents besoins. Sur le plan du développement, les équipes travaillent avec des environnements tels que Visual Studio Code, Git et GitHub pour la gestion du code source, ainsi que des plateformes cloud comme AWS ou Azure pour l'hébergement des applications. Pour la gestion de projets, des outils comme Jira ou Trello sont utilisés au sein du PMO. La communication interne repose sur des solutions telles que Slack ou Microsoft Teams. Enfin, les formations de l'Akieni Academy s'appuient sur des plateformes d'apprentissage en ligne (LMS) permettant de suivre la progression des apprenants à distance.

1.2 Le sujet

1.2.1 Contexte du sujet

Le numérique occupe aujourd'hui une place centrale dans l'organisation des services professionnels. Les plateformes numériques ont remplacé progressivement les méthodes traditionnelles de recherche de prestataires, offrant plus de transparence et d'efficacité. Cependant, à Brazzaville, de nombreux professionnels exercent dans des domaines variés comme l'artisanat, les services techniques, les prestations intellectuelles ou les services à la personne, mais ils restent peu visibles faute d'une plateforme numérique adaptée.

De leur côté, les particuliers, les entreprises et les institutions ont du mal à trouver rapidement un prestataire compétent, disponible et de confiance. Les échanges reposent encore largement sur le bouche-à-oreille, sans aucune garantie de qualité ni de sécurité financière.

« SALISA » naît de ce constat. Il s'agit d'une application cross-plateforme conçue pour faciliter la mise en relation entre prestataires et clients, en intégrant un système de vérification des profils, un mécanisme de paiement sécurisé et un outil de notation des prestations. Le nom *SALISA*, emprunté au lingala, a une double signification : pour le prestataire, *ko salisa* signifie "aider" et pour le demandeur, *salisa* signifie "faire faire" (un travail).

1.2.2 Problématique du sujet

Malgré la présence de nombreux prestataires de services à Brazzaville, il n'existe pas de mécanisme fiable permettant de les mettre en relation avec les clients. Les échanges se font de façon informelle, sans garantie de qualité, de ponctualité ni de sécurité financière pour les parties concernées.

La problématique principale de ce travail peut être formulée ainsi :

Comment concevoir et mettre en œuvre une application cross-plateforme efficace permettant de connecter des prestataires et des clients, tout en garantissant la fiabilité des services, la sécurité des transactions et une expérience utilisateur adaptée au contexte local ?

Cette problématique soulève plusieurs enjeux : la gestion de la confiance entre utilisateurs, l'optimisation du choix des prestataires grâce à un mécanisme de mise en correspondance intelligent, la sécurisation des paiements, et l'accessibilité de la solution sur des appareils variés.

1.2.3 Intérêts du sujet

L'intérêt de ce travail se situe à plusieurs niveaux.

Sur le plan économique : la plateforme permet de dynamiser le marché des services en facilitant l'accès aux opportunités pour les prestataires et en réduisant les difficultés de recherche pour les clients. Elle contribue à la création de valeur économique locale et à l'insertion professionnelle.

Sur le plan social : « SALISA » favorise l'inclusion numérique en offrant à un large public la possibilité d'accéder à des services fiables et de faire connaître leurs compétences. Elle encourage également les transactions formelles entre citoyens.

Sur le plan technologique : le travail intègre des solutions modernes comme la mise en correspondance intelligente (matching), la géolocalisation par quartier et le paiement mobile via MTN MoMo et Airtel Money, ce qui renforce sa pertinence dans un contexte numérique en développement.

Sur le plan académique : ce travail est une application concrète des enseignements du génie logiciel, notamment l'analyse des besoins, la modélisation UML, la conception de systèmes distribués et le développement d'applications selon la démarche **2TUP**.

1.3 Concepts liés au sujet

La compréhension de ce travail nécessite la maîtrise de plusieurs concepts clés, définis ci-après.

Application cross-plateforme : désigne une application conçue pour fonctionner sur plusieurs types de terminaux et de systèmes d'exploitation (web, Android, iOS) à partir d'une seule base de code, ce qui réduit les coûts et les délais de développement.

Mise en relation : processus qui consiste à connecter une offre de service proposée par un prestataire à une demande exprimée par un client, en s'appuyant sur des critères de compatibilité définis.

Matching : mécanisme qui identifie automatiquement les prestataires les plus adaptés à une demande, en tenant compte des compétences, de la localisation géographique, de la disponibilité, du budget et des évaluations reçues.

Système de confiance : ensemble de mécanismes tels que les notes, les avis, la vérification d'identité et les badges de certification, qui permettent de s'assurer de la fiabilité des utilisateurs et de la qualité des services proposés.

Paiement mobile : solution de paiement électronique très utilisée en Afrique, qui permet d'effectuer des transactions financières via un téléphone mobile, sans avoir besoin d'un compte bancaire. Au Congo, les deux opérateurs de paiement mobile sont MTN MoMo et Airtel Money.

API (Application Programming Interface) : interface de programmation qui permet à deux applications de communiquer entre elles en échangeant des données selon un protocole défini. Dans le cadre de « SALISA », les API de MTN MoMo et d'Airtel Money sont utilisées pour intégrer

le paiement mobile directement dans la plateforme.

Progressive Web Application (PWA) : application web qui offre une expérience proche d'une application mobile classique, avec un fonctionnement possible hors connexion, l'installation sur l'écran d'accueil et des notifications. Elle ne nécessite pas de téléchargement depuis un store d'applications.

UML (Unified Modeling Language) : langage de modélisation graphique utilisé en génie logiciel pour représenter les différents aspects d'un système informatique, comme les cas d'utilisation, les classes, les séquences ou le déploiement.

2TUP (Two Tracks Unified Process) : processus de développement logiciel dérivé du Processus Unifié (UP), qui organise la conception selon deux axes parallèles : la branche fonctionnelle (besoins métier) et la branche technique (contraintes d'architecture), qui fusionnent en phase de réalisation.

Géolocalisation : technique qui permet de situer un utilisateur ou un service dans un espace géographique, afin de faciliter la mise en relation de proximité entre prestataires et clients.

Deuxième Partie

ANALYSE ET CONCEPTION

Chapitre I : Étude préliminaire et Méthode

1.1 Étude de l'existant

1.1.1 Description des activités (situation actuelle)

Akieni est une société spécialisée dans la transformation numérique et les solutions logicielles, basée à Brazzaville. Elle propose des formations aux métiers du numérique, de six à neuf mois, aux jeunes de 16 à 30 ans, dans des domaines comme le développement web, la science des données et la conception d'interfaces. Elle accompagne aussi des entreprises et des institutions dans leurs projets de transformation numérique, notamment en leur fournissant des outils digitaux adaptés à leurs besoins.

Cependant, malgré ces activités, la mise en relation entre les prestataires de services professionnels et les clients se fait encore de manière informelle. Les prestataires se font connaître par le bouche-à-oreille, les contacts téléphoniques directs ou les recommandations de proches. Les clients, de leur côté, n'ont aucun outil pour comparer les offres, vérifier les compétences des prestataires ou sécuriser leurs paiements.

1.1.2 Critique de l'existant (limites)

Bien qu'Akieni soit un acteur important de la transformation numérique au Congo, il n'existe, à ce jour, aucune application ou plateforme dédiée à la mise en relation entre prestataires de services et clients au sein de la structure. Toute la gestion de cette mise en relation repose sur des échanges informels, ce qui engendre plusieurs limites importantes :

- les prestataires n'ont aucun moyen simple de se rendre visibles auprès des clients potentiels, leur notoriété dépend entièrement de leur réseau personnel ;
- les clients ne disposent d'aucun outil pour évaluer la qualité ou la disponibilité d'un prestataire avant de le contacter ;
- les transactions se font sans encadrement : il n'existe aucun système de paiement sécurisé, aucun mécanisme pour résoudre les litiges et aucune garantie en cas de prestation non réalisée ;
- il n'y a aucune traçabilité des missions réalisées ni de système de notation permettant de mesurer la satisfaction des clients ;
- le processus de recherche d'un prestataire est long, peu fiable et source d'incertitude pour les deux parties.

Ces limites montrent clairement que la situation actuelle nécessite une solution numérique structurée. C'est pourquoi, dans le point suivant, nous allons proposer plusieurs solutions envisageables pour y répondre.

1.1.3 Proposition des solutions

Face aux limites identifiées, il est apparu nécessaire d'explorer différentes pistes pour y répondre. Après analyse et discussion au sein de notre équipe, trois solutions ont été envisagées, chacune présentant ses propres caractéristiques, ses avantages et ses inconvénients. Ces solutions sont présentées de façon équitable afin de permettre au lecteur de se faire une idée objective de chacune d'elles.

a- Solution 1 : portail web de mise en relation avec annuaire de profils et messagerie intégrée (PHP, MySQL, Bootstrap)

Il s'agit de concevoir un portail web permettant aux prestataires de publier un profil détaillé regroupant leurs compétences, leurs tarifs et leur zone d'intervention, et aux clients de les rechercher et de les contacter directement via un système de messagerie intégré. Cette approche repose sur des technologies web éprouvées, telles que PHP côté serveur, MySQL pour la base de données et Bootstrap pour l'interface. Elle vise avant tout à offrir une visibilité numérique simple aux prestataires qui n'en ont aucune aujourd'hui.

Avantages :

- accessible depuis tous les terminaux (smartphone, tablette, ordinateur) via un navigateur, sans téléchargement ;
- mise en place rapide et coût de développement modéré, grâce à des technologies simples et bien documentées ;
- interface intuitive, facile à prendre en main même pour les utilisateurs peu expérimentés ;
- bonne visibilité des prestataires grâce aux profils détaillés et référençables par les moteurs de recherche.

Inconvénients :

- pas de gestion intégrée des paiements ni de mécanisme de sécurisation des fonds ;
- pas de système de mise en correspondance automatique entre prestataires et clients ;
- les litiges ne peuvent pas être gérés de façon structurée depuis la plateforme.

b- Solution 2 : développement d'une application web cross-plateforme (PWA-like) avec matching intelligent et paiement sécurisé

Il s'agit de développer une application web, accessible depuis tous les terminaux sans installation obligatoire. Elle intègre des fonctionnalités complètes, à savoir la gestion des profils, la publication de missions, la messagerie, la mise en correspondance intelligente entre prestataires et clients, le règlement des missions via MTN MoMo ou Airtel Money, et le système de notation. C'est la solution que nous avons retenue pour « SALISA ».

Avantages :

- une seule base de code compatible avec tous les terminaux, ce qui réduit les coûts et les délais de développement ;
- aucun téléchargement nécessaire, l'application est accessible directement via un navigateur ;
- fonctionne correctement même sur des connexions internet lentes (2G/3G), grâce à son architecture mobile-first ;
- intègre le paiement mobile via MTN MoMo et Airtel Money, adapté au contexte local [4] ;
- algorithme de mise en correspondance pour aider les clients à trouver le bon prestataire rapidement ;
- système de notification multi-canaux pour tenir les utilisateurs informés en temps réel.

Inconvénients :

- nécessite une connexion internet pour bénéficier de toutes les fonctionnalités ;
- certaines fonctionnalités natives des terminaux peuvent être légèrement plus limitées que dans une application mobile native.

c- Solution 3 : développement d'une application mobile native de mise en relation entre prestataires et clients (React Native)

Il s'agit de développer une application mobile native, conçue spécifiquement pour les systèmes Android et iOS, en s'appuyant sur le framework React Native qui permet de partager une grande partie du code entre les deux plateformes tout en accédant aux fonctionnalités propres à chaque terminal. Cette application intégrerait toutes les fonctionnalités de mise en relation, de paiement et de notification, en tirant pleinement parti des capacités matérielles des smartphones, notamment le GPS, l'appareil photo et les notifications système.

Avantages :

- expérience utilisateur optimale sur chaque système, grâce à l'accès complet aux fonctionnalités natives du terminal;
- performances élevées, même pour les animations et les interactions complexes;
- notifications push natives, très efficaces pour alerter les utilisateurs en temps réel;
- fonctionnement hors ligne poussé, grâce au stockage local des données sur le terminal.

Inconvénients :

- coût de développement plus élevé car il faut maintenir deux versions distinctes, l'une pour Android et l'autre pour iOS, même avec React Native;
- l'utilisateur doit télécharger l'application depuis un store, ce qui constitue une barrière à l'adoption;
- les mises à jour doivent être soumises et validées par les stores avant d'être disponibles.

1.1.4 Choix de la solution

Notre choix se porte sur la **solution 2**.

1.2 Méthode

En génie logiciel, une **méthode** désigne l'association d'un langage de modélisation et d'un processus de développement. Le langage de modélisation sert à représenter le système de façon graphique et structurée, tandis que le processus de développement organise les étapes à suivre pour le concevoir et le réaliser. Pour « SALISA », nous avons couplé **UML** à un processus unifié [5]. Parmi les processus unifiés qui existent, comme le RUP (*Rational Unified Process*) ou l'OpenUP (*Open Unified Process*), notre choix s'est porté sur le **2TUP** (*Two Tracks Unified Process*), qui est le mieux adapté à la complexité et aux contraintes de notre projet.

1.2.1 Le langage de modélisation

1.2.1.1 Présentation d'UML

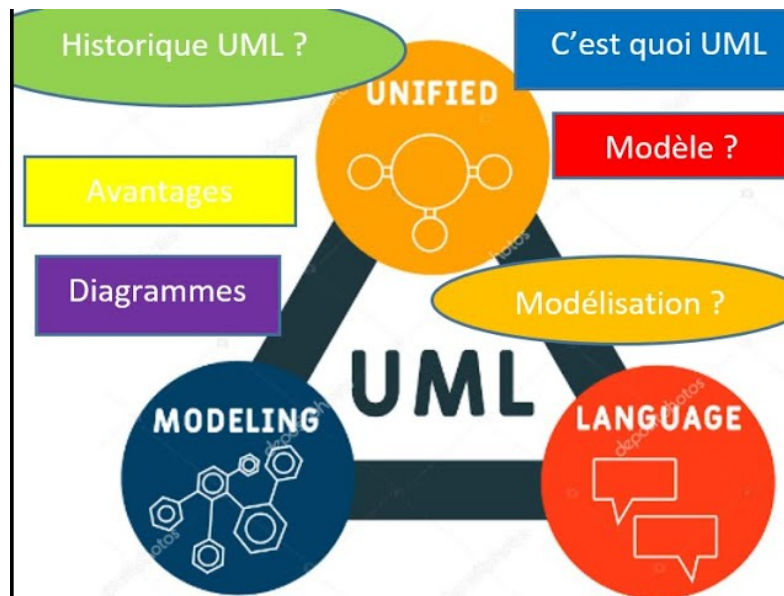


Figure 2 : Les concepts clés du langage UML

UML (*Unified Modeling Language*, ou Langage de Modélisation Unifié) est un langage graphique standardisé par l'OMG (*Object Management Group*) depuis 1997 [1]. Il constitue aujourd'hui la référence mondiale pour la modélisation des systèmes logiciels orientés objet.

L'objectif d'UML est de fournir un langage visuel commun qui permet aux équipes de travail, qu'ils soient analystes, concepteurs, développeurs ou clients, de communiquer clairement sur la structure et le comportement d'un système. Il peut s'intégrer à toute démarche de développement, itérative ou non.

Pour « SALISA », UML présente plusieurs atouts : ses notations sont reconnues et documentées dans le monde entier, facilitant la compréhension et la maintenance du système ; il permet de représenter aussi bien la vue statique que la vue dynamique du système [2] ; il est intégralement utilisé par le processus 2TUP pour produire ses livrables de modélisation ; et ses diagrammes servent de lien clair entre les besoins exprimés et le travail réalisé par les développeurs.

1.2.1.2 Les types de diagrammes UML

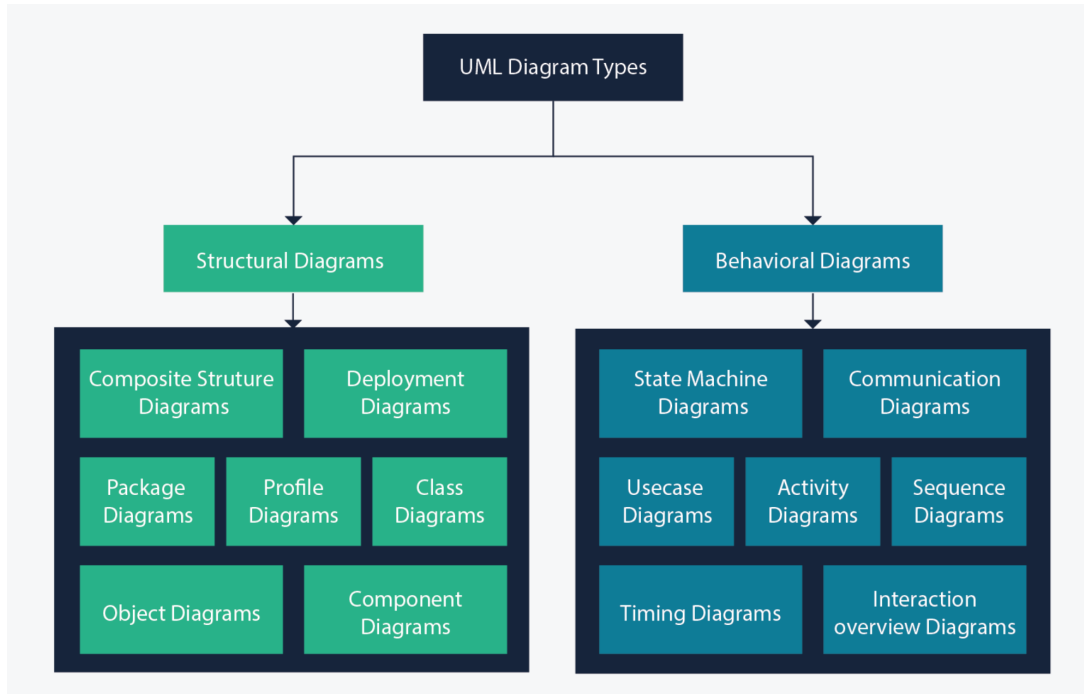


Figure 3 : Les types de diagrammes UML

La version 2.x d'UML propose **quatorze types de diagrammes**, répartis en deux grandes familles [1].

a- Diagrammes structurels (vue statique)

Ces diagrammes décrivent la structure permanente du système. On y trouve notamment le diagramme de classes, qui représente les classes du système, leurs attributs, leurs méthodes et les relations qui les unissent ; le diagramme d'objets, qui montre des instances concrètes des classes à un moment précis ; le diagramme de composants, qui décrit l'architecture physique du logiciel ; le diagramme de déploiement, qui représente la répartition des composants sur les équipements matériels ; le diagramme de paquetages, qui organise les éléments du modèle en groupes logiques ; le diagramme de structure composite ; et le diagramme de profils.

b- Diagrammes comportementaux (vue dynamique)

Ces diagrammes décrivent le comportement du système dans le temps. Parmi eux, on trouve entre autres le diagramme de cas d'utilisation, qui identifie les acteurs et les fonctionnalités qu'ils peuvent utiliser ; le diagramme de séquence, qui montre les échanges entre objets dans le temps ; le diagramme de communication ; le diagramme d'activités, qui représente un enchaînement d'actions et de décisions dans un processus ; le diagramme

d'états-transitions; le diagramme de vue d'ensemble des interactions; et le diagramme de temps.

Pour «SALISA», nous utiliserons principalement le **diagramme de contexte statique**, le **diagramme de cas d'utilisation**, le **diagramme de séquence**, le **diagramme d'activités**, le **diagramme de classes** et le **diagramme de déploiement**.

1.2.2 Le processus de développement

1.2.2.1 Présentation du Processus Unifié (UP)

Le **Processus Unifié** (*Unified Process*, abrégé **UP**) est un cadre de développement logiciel itératif et incrémental, formalisé par Ivar Jacobson, Grady Booch et James Rumbaugh à la fin des années 1990 [3]. Sa version commerciale la plus connue est le *Rational Unified Process* (RUP).

L'UP repose sur quatre principes fondamentaux : il est itératif et incrémental, centré sur l'architecture, piloté par les cas d'utilisation, et orienté vers la réduction des risques. Il s'organise en quatre phases : l'Inception pour définir le périmètre du travail ; l'Élaboration pour capturer les besoins et concevoir l'architecture ; la Construction pour développer les fonctionnalités ; et la Transition pour déployer le produit.

1.2.2.2 Présentation du processus 2TUP

Le **2TUP** (*Two Tracks Unified Process*) est une adaptation du Processus Unifié, proposée par Pascal Roques et Franck Vallée dans leur ouvrage *UML en action* [5]. Sa particularité est son organisation en deux branches parallèles qui se rejoignent en phase de réalisation.

a- Branche fonctionnelle (What)

Elle recueille et formalise ce que les utilisateurs attendent du système. On y trouve entre autres la capture des besoins fonctionnels, la modélisation des cas d'utilisation, la description textuelle et les diagrammes de séquence des cas d'utilisation prioritaires, ainsi que la modélisation du domaine métier à travers le diagramme de classes.

b- Branche technique (How)

Elle prend en charge les contraintes d'infrastructure et d'architecture liées au contexte du projet. Elle couvre la capture des besoins techniques et des contraintes non fonctionnelles, la définition de l'architecture logicielle et matérielle, le choix des technologies et des outils, ainsi que le prototypage de l'architecture technique.

c- Branche de réalisation (fusion)

Les deux branches précédentes se rejoignent pour produire le système final. Cette phase comprend la conception préliminaire qui intègre les modèles fonctionnels dans l'architecture technique, la conception détaillée qui spécifie précisément les composants et les interfaces, ainsi que le codage, les tests et l'intégration pour aboutir à la livraison du produit.

La figure 4 illustre la structure en forme de « Y » caractéristique du processus 2TUP.

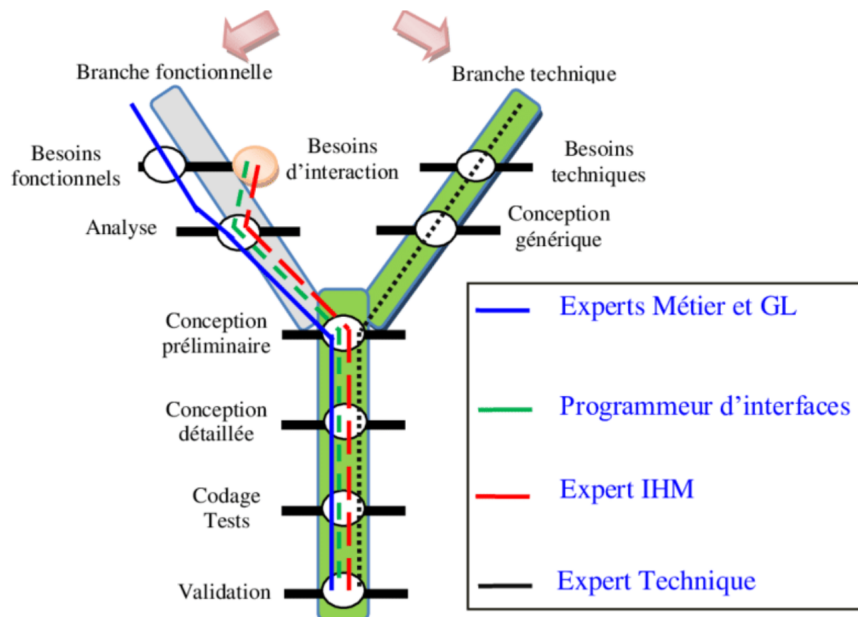


Figure 4 : Structure en Y du processus 2TUP

1.2.2.3 2TUP et UML

Le processus 2TUP et le langage UML fonctionnent ensemble : 2TUP utilise UML comme unique langage de modélisation tout au long de ses phases, et chaque livrable produit est un diagramme UML [5]. Cette cohérence garantit une bonne traçabilité entre les besoins exprimés et les composants réalisés.

Le tableau 1 ci-dessous résume la correspondance entre les phases du 2TUP et les diagrammes UML mobilisés pour « SALISA ».

Phase 2TUP	Branche	Diagrammes UML utilisés
Capture des besoins fonctionnels	Fonctionnelle	Diagramme de contexte statique, Diagramme de cas d'utilisation
Spécification détaillée	Fonctionnelle	Diagramme de séquence, Diagramme d'activités
Capture des besoins techniques	Technique	Diagramme de déploiement
Conception architecturale	Réalisation	Diagramme de composants, Diagramme de déploiement
Conception du domaine	Réalisation	Diagramme de classes, Diagramme d'objets

Tableau 1 : Correspondance entre les phases 2TUP et les diagrammes UML utilisés

« SALISA » combine des besoins fonctionnels nombreux, comme l'inscription des utilisateurs, la mise en correspondance, le paiement ou la messagerie, et des contraintes techniques fortes liées au contexte local, telles que le paiement mobile via MTN MoMo et Airtel Money [4], une connexion parfois limitée ou une architecture distribuée. Le 2TUP permet de traiter ces deux dimensions en parallèle, de façon rigoureuse et coordonnée, ce qui en fait le processus idéal pour notre projet.

Chapitre II : Analyse et Conception

2.1 Analyse du besoin

2.1.1 Besoins Fonctionnels

Les besoins fonctionnels représentent l'ensemble des fonctionnalités que « SALISA » doit offrir à ses utilisateurs. Nous les avons identifiés à partir de l'analyse des problèmes rencontrés dans la mise en relation manuelle entre prestataires et clients. Ces besoins sont les suivants :

- inscription via un numéro de téléphone, avec validation par code OTP ;
- connexion et déconnexion sécurisées ;
- choix du rôle lors de l'inscription (prestataire ou demandeur) ;
- création et gestion du profil prestataire (informations, portfolio, tarifs, disponibilité, catégories de services) ;
- vérification d'identité et obtention de badges de certification ;
- publication de missions par le demandeur ;
- recherche et filtrage de prestataires par catégorie, arrondissement, disponibilité et budget ;
- mise en correspondance intelligente (matching) entre missions et prestataires ;
- messagerie directe entre demandeur et prestataire ;
- envoi et acceptation de devis structurés ;
- paiement sécurisé via MTN MoMo et Airtel Money [4] ;
- validation de la mission terminée et confirmation du paiement au prestataire ;
- notation et dépôt d'avis après chaque mission réalisée ;
- notifications multi-canaux (push PWA, e-mail, SMS) ;
- tableau de bord de modération pour les administrateurs.

2.1.2 Besoins Techniques

Les besoins techniques désignent l'ensemble des contraintes non fonctionnelles auxquelles un système doit se conformer, qu'il s'agisse de performances, de sécurité, de disponibilité ou de compatibilité avec l'environnement d'exécution. Pour « SALISA », ces contraintes sont directement liées au contexte congolais, et se déclinent de la façon suivante :

- architecture Progressive Web Application (PWA) [7], mobile-first, utilisable depuis tous les terminaux sans installation ;
- compatibilité avec les réseaux à faible débit (2G/3G) pour garantir l'accès dans les zones peu couvertes ;
- fonctionnement hors ligne partiel pour la consultation des profils et des missions déjà chargés ;
- sécurité des échanges par protocoles HTTPS et WSS, authentification par JWT et OTP ;
- performance : temps de réponse des API inférieur à 2 secondes pour 95% des requêtes ;
- disponibilité cible de 99,5% ;
- intégration des API de paiement mobile : MTN MoMo et Airtel Money [4] ;
- base de données relationnelle (PostgreSQL) [6] avec cache Redis pour les sessions et les résultats fréquents ;
- messagerie en temps réel via WebSocket ;
- stockage des fichiers (photos de profil, pièces jointes) sur un service cloud externe.

2.2 Conception du système

2.2.1 Spécifications fonctionnelles

2.2.1.1 Identification des acteurs

Un acteur représente toute entité externe qui interagit avec le système « SALISA » [3]. Notre système comprend deux catégories d'acteurs : les acteurs humains et les acteurs systèmes.

a- Acteurs humains

Nous avons identifié quatre principaux acteurs humains :

- **Visiteur** : utilisateur non encore enregistré sur la plateforme. Son unique interaction avec « SALISA » est l'inscription, qui lui permet de créer un compte et de choisir son rôle ;
- **Demandeur** : utilisateur connecté qui cherche et embauche des prestataires. Il publie des missions, contacte des prestataires, paie et note les prestations ;
- **Prestataire** : utilisateur connecté qui propose ses services. Il crée son profil, répond aux missions et envoie des devis ;
- **Administrateur** : utilisateur chargé d'administrer la plateforme, de vérifier les identités, de gérer les signalements et de configurer le système.

b- Acteurs systèmes

En modélisation UML, on désigne par acteur système tout composant logiciel extérieur au système étudié qui interagit avec lui par l'échange de données ou de services. Ces acteurs sont représentés avec le stéréotype `<<system>>` pour les distinguer des acteurs humains. Nous en avons identifié deux pour « SALISA » :

- **Service de paiement** : regroupe les APIs de MTN MoMo et d'Airtel Money. « SALISA » leur envoie des requêtes de paiement et en reçoit des confirmations de transaction [4] ;
- **Service de notifications** : regroupe le service d'e-mail transactionnel et l'API SMS. « SALISA » lui envoie des instructions pour délivrer les codes OTP, les alertes et les messages aux utilisateurs.

2.2.1.2 Diagramme de contexte statique

Le diagramme de contexte statique présente « SALISA » au centre et montre toutes les entités externes qui interagissent avec lui. La figure 5 illustre ce diagramme.

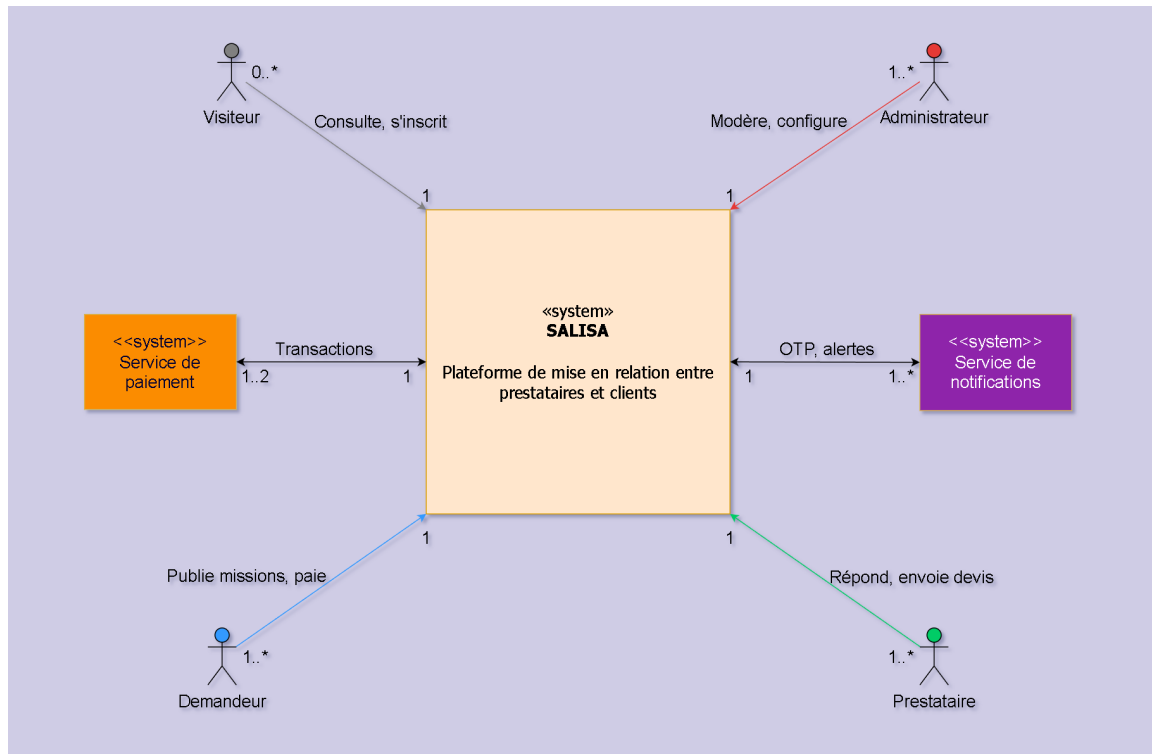


Figure 5 : Diagramme de contexte statique de « SALISA »

Au centre se trouve le système « SALISA » représenté par un rectangle avec le stéréotype «<<system>>». Autour, les quatre acteurs humains (symbole stickman UML) interagissent avec lui : le Visiteur consulte et s'inscrit sur la plateforme ; le Demandeur publie des missions et paie ; le Prestataire crée son profil, répond et envoie des devis ; l'Administrateur modère et configure la plateforme. Les deux acteurs systèmes (rectangles avec stéréotype «<<system>>») sont : le Service de paiement, qui gère les transactions de paiement mobile (MTN MoMo et Airtel Money), et le Service de notifications, qui délivre les alertes e-mail et SMS aux utilisateurs.

2.2.1.3 Identification des cas d'utilisation

Un cas d'utilisation décrit une interaction entre un acteur et le système pour accomplir un objectif précis. Pour « SALISA », nous avons recensé 35 cas d'utilisation. Compte tenu de leur nombre et de la diversité des domaines qu'ils couvrent, les regrouper dans un seul diagramme de cas d'utilisation aurait rendu ce dernier illisible et peu exploitable. Nous avons donc opté pour une organisation en neuf packages fonctionnels cohérents, conformément aux recommandations de la norme UML 2 pour les systèmes de taille significative [5]. Chaque package regroupe les cas d'utilisation qui relèvent d'un même domaine métier. Les tableaux 2 à 10 présentent le détail de chacun de ces packages.

Package P1 : Authentification	
Cas d'utilisation	Définition
UC-01	s'inscrire via numéro de téléphone (Visiteur)
UC-02	valider le code OTP (Visiteur)
UC-03	choisir son rôle lors de l'inscription (Visiteur)
UC-04	se connecter (Demandeur, Prestataire, Administrateur)
UC-05	se déconnecter (Demandeur, Prestataire, Administrateur)

Tableau 2 : Cas d'utilisation du package P1 : Authentification

Package P2 : Profil prestataire	
Cas d'utilisation	Définition
UC-06	compléter son profil prestataire (Prestataire)
UC-07	gérer son portfolio (Prestataire)
UC-08	définir ses tarifs et sa disponibilité (Prestataire)
UC-09	gérer ses catégories et compétences (Prestataire)
UC-10	soumettre une demande de vérification d'identité (Prestataire, Administrateur)

Tableau 3 : Cas d'utilisation du package P2 : Profil prestataire

Package P3 : Gestion des missions	
Cas d'utilisation	Définition
UC-11	publier une mission (Demandeur)
UC-12	utiliser l'assistant IA pour la description de la mission (Demandeur)
UC-13	modifier ou clore une mission publiée (Demandeur)

Tableau 4 : Cas d'utilisation du package P3 : Gestion des missions

Package P4 : Recherche et matching	
Cas d'utilisation	Définition
UC-14	rechercher un prestataire (Demandeur)
UC-15	filtrer les résultats de recherche (Demandeur)
UC-16	consulter le score de matching (Demandeur)
UC-17	recevoir des suggestions proactives de prestataires (Demandeur)
UC-18	consulter le profil d'un prestataire (Demandeur)

Tableau 5 : Cas d'utilisation du package P4 : Recherche et matching

Package P5 : Messagerie et devis	
Cas d'utilisation	Définition
UC-19	envoyer un message (Demandeur, Prestataire)
UC-20	envoyer un devis (Prestataire)
UC-21	accepter un devis (Demandeur)
UC-22	refuser ou demander une modification de devis (Demandeur)

Tableau 6 : Cas d'utilisation du package P5 : Messagerie et devis

Package P6 : Paiement	
Cas d'utilisation	Définition
UC-23	régler une mission via MTN MoMo ou Airtel Money (Demandeur, Service de paiement)
UC-24	choisir MTN MoMo comme moyen de paiement (Demandeur)
UC-25	choisir Airtel Money comme moyen de paiement (Demandeur)
UC-26	valider la mission terminée (Demandeur)
UC-27	demander un retrait (Prestataire)

Tableau 7 : Cas d'utilisation du package P6 : Paiement

Package P7 : Notation et réputation	
Cas d'utilisation	Définition
UC-28	laisser un avis après mission (Demandeur, Prestataire)
UC-29	signaler un avis abusif (Demandeur, Prestataire)

Tableau 8 : Cas d'utilisation du package P7 : Notation et réputation

Package P8 : Notifications	
Cas d'utilisation	Définition
UC-30	recevoir des notifications multi-canaux (Demandeur, Prestataire, Service de notifications)
UC-31	configurer ses préférences de notification (Demandeur, Prestataire)

Tableau 9 : Cas d'utilisation du package P8 : Notifications

Package P9 : Administration	
Cas d'utilisation	Définition
UC-32	accéder au tableau de bord de modération (Administrateur)
UC-33	valider ou rejeter une vérification d'identité (Administrateur)
UC-34	gérer les signalements (Administrateur)
UC-35	suspendre un compte utilisateur (Administrateur)

Tableau 10 : Cas d'utilisation du package P9 : Administration

2.2.1.4 Relation entre les cas d'utilisation

Les relations entre cas d'utilisation permettent de structurer et de clarifier les dépendances entre les fonctionnalités du système [5]. On en distingue deux types principaux. La relation **«include»** indique qu'un cas d'utilisation est toujours exécuté dans le cadre d'un autre : par exemple, UC-01 (s'inscrire) inclut systématiquement UC-02 (valider le code OTP), car on ne peut pas créer de compte sans valider le code reçu par SMS; de même, UC-21 (accepter un devis) inclut UC-23 (régler une mission via MTN MoMo ou Airtel Money), car l'acceptation d'un devis déclenche toujours le processus de paiement. La relation **«extend»**, quant à elle, indique qu'un cas d'utilisation est exécuté de façon optionnelle dans certaines conditions : par exemple, UC-11 (publier une mission) peut optionnellement étendre vers UC-12 (utiliser l'assistant IA pour la description de la mission), selon le choix du demandeur.

Pour illustrer ces dépendances avec précision, nous avons d'abord représenté chaque package dans son propre diagramme de cas d'utilisation. Ces neuf diagrammes détaillés permettent de visualiser clairement, pour chaque domaine fonctionnel, les acteurs impliqués et les relations entre les cas d'utilisation. Les figures 6 à 14 illustrent ces neuf diagrammes.

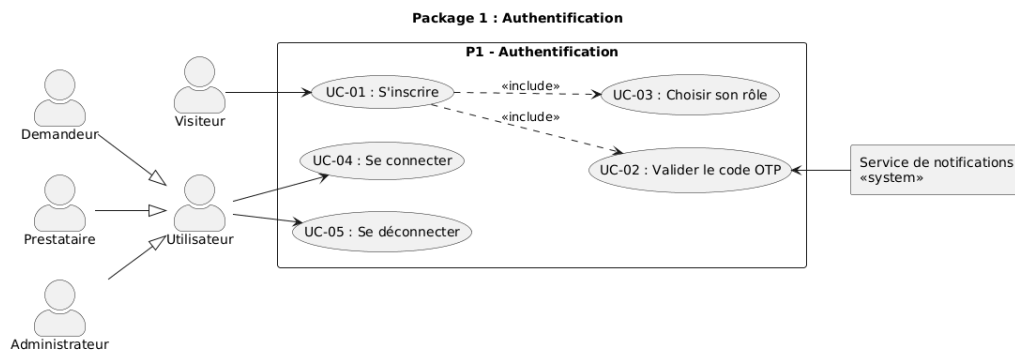


Figure 6 : Diagramme de cas d'utilisation - P1 : Authentification

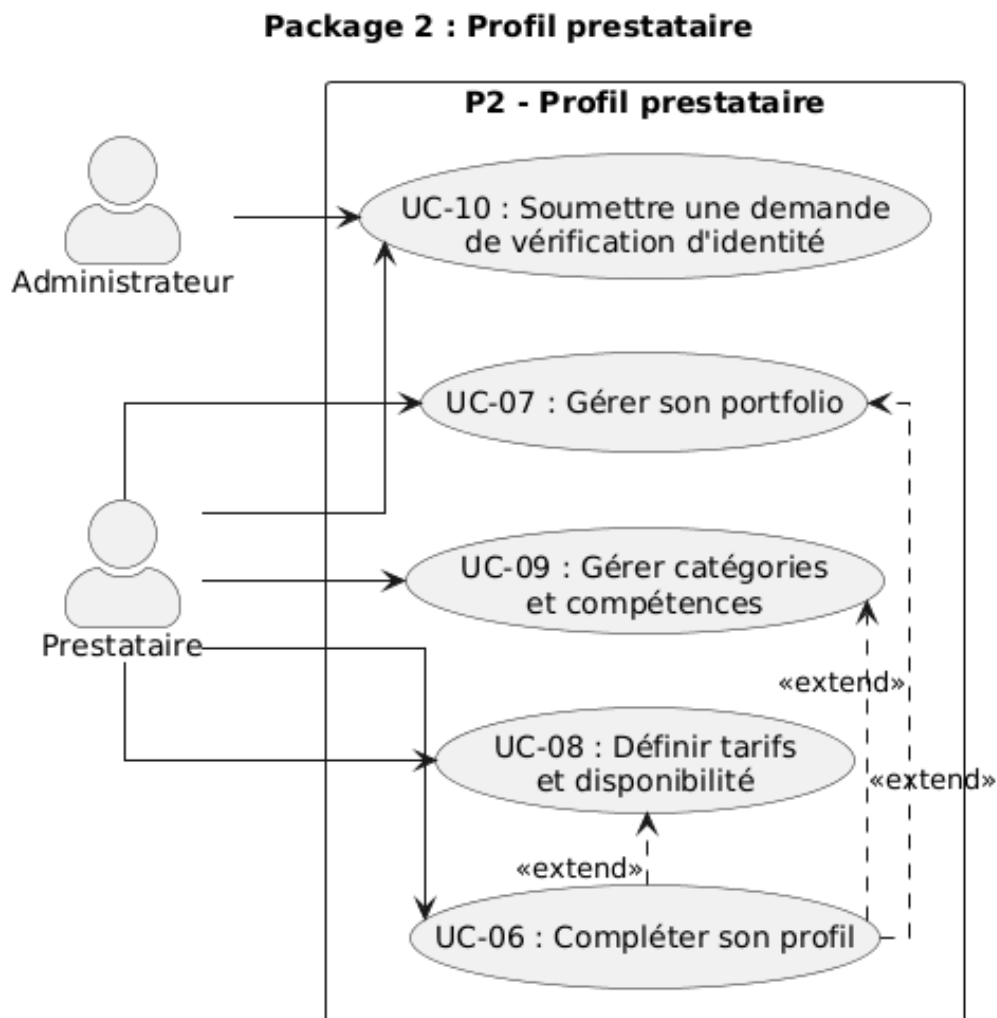


Figure 7 : Diagramme de cas d'utilisation - P2 : Profil prestataire

Package 3 : Gestion des missions

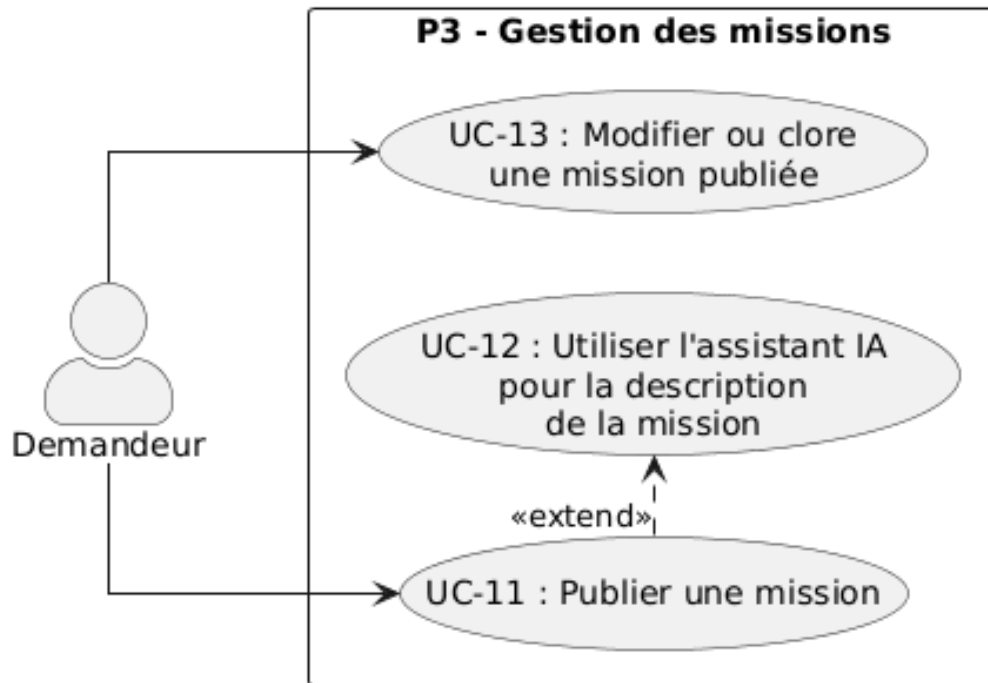


Figure 8 : Diagramme de cas d'utilisation - P3 : Gestion des missions

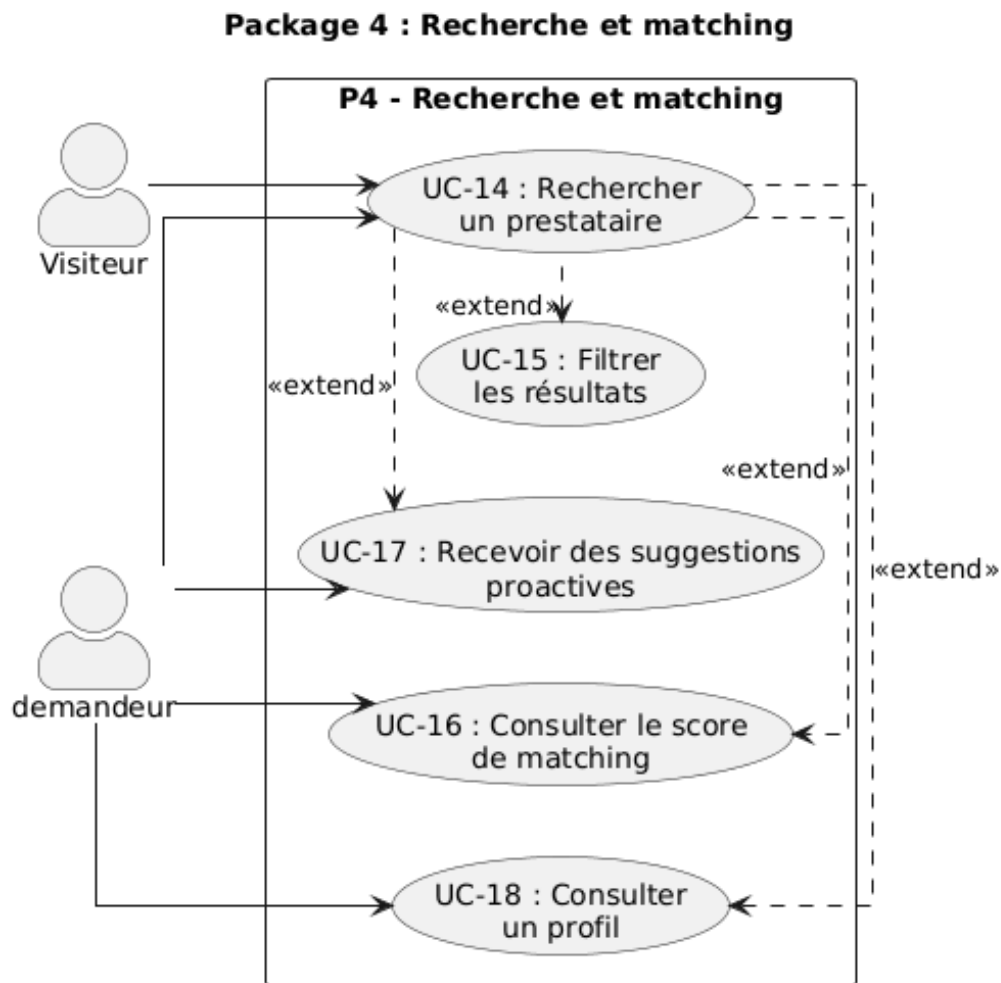


Figure 9 : Diagramme de cas d'utilisation - P4 : Recherche et matching

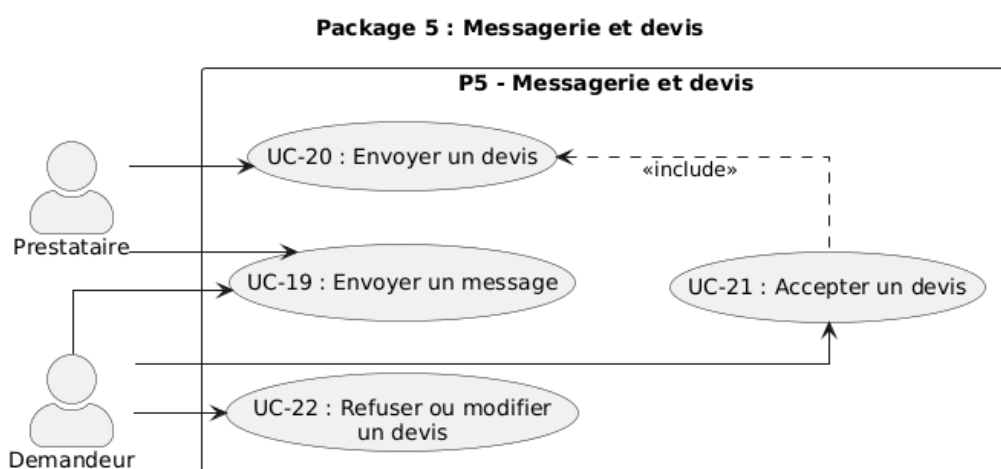


Figure 10 : Diagramme de cas d'utilisation - P5 : Messagerie et devis

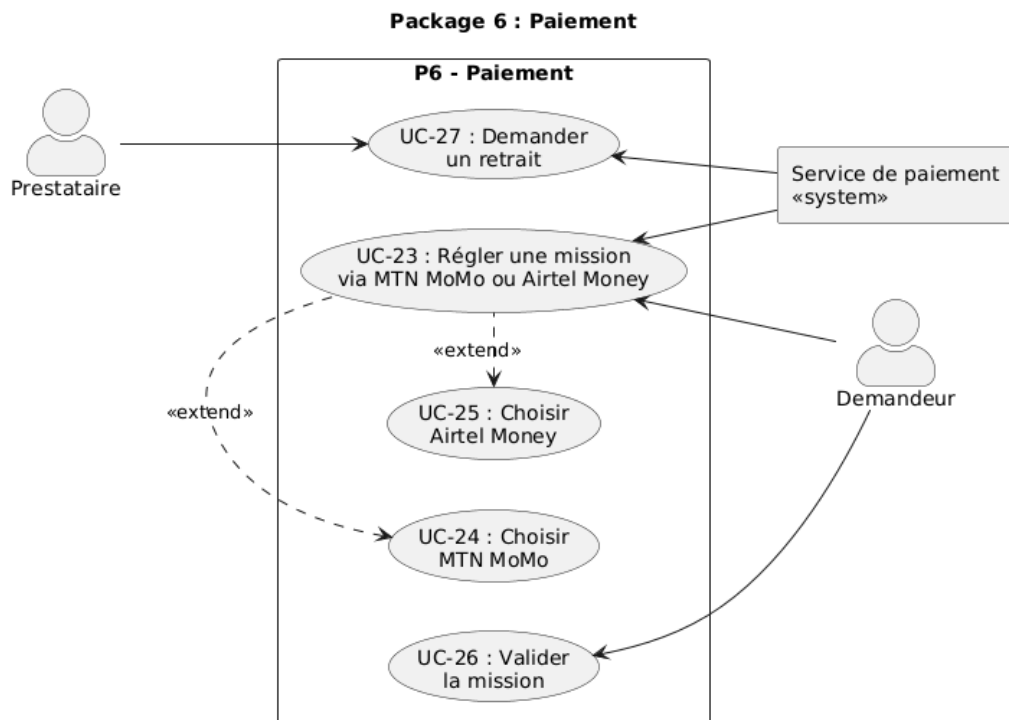


Figure 11 : Diagramme de cas d'utilisation - P6 : Paiement

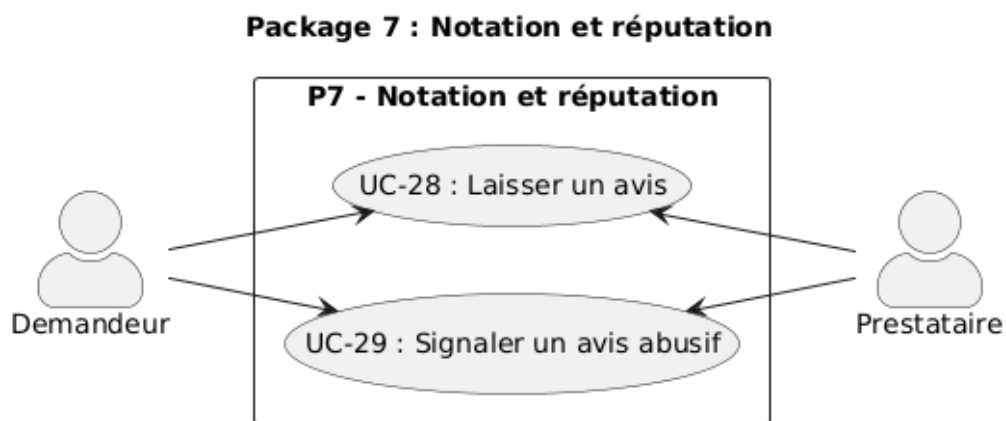


Figure 12 : Diagramme de cas d'utilisation - P7 : Notation et réputation

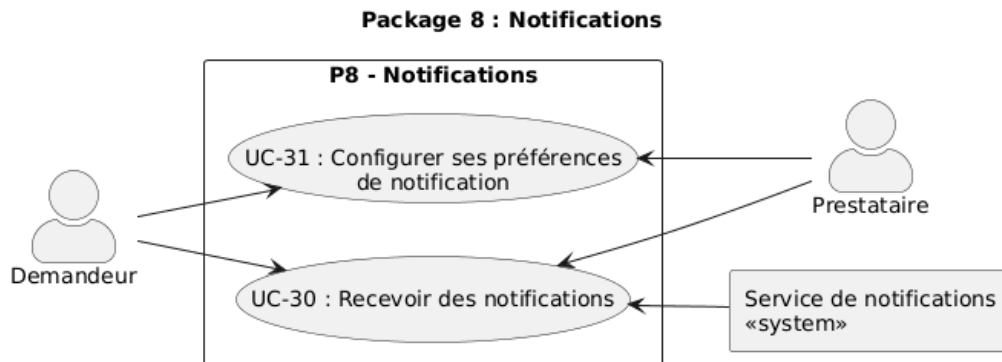


Figure 13 : Diagramme de cas d'utilisation - P8 : Notifications

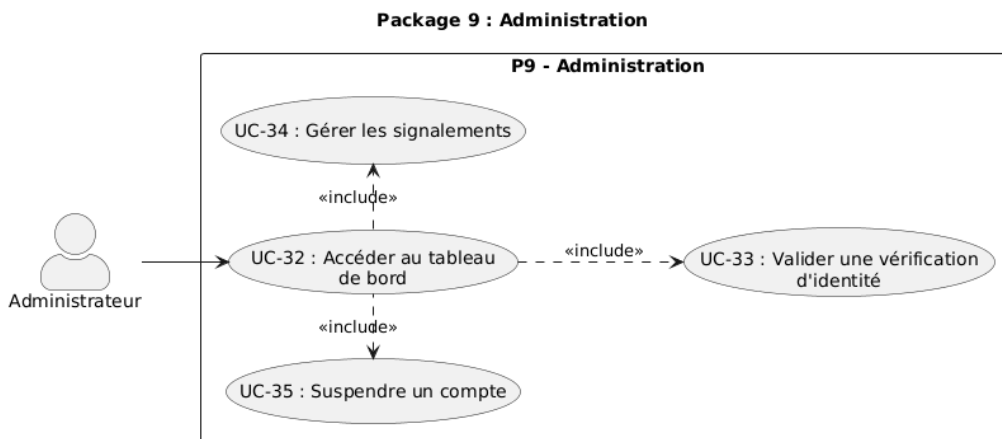


Figure 14 : Diagramme de cas d'utilisation - P9 : Administration

Une fois les neuf packages détaillés individuellement, il est utile de disposer d'une vue d'ensemble du système. C'est le rôle du diagramme de packages présenté à la figure 15. Ce diagramme montre les neuf domaines fonctionnels de « SALISA » et leurs relations avec les acteurs. Il met notamment en évidence une dépendance commune à tous les packages : chacun d'entre eux requiert que l'utilisateur soit préalablement authentifié, ce qui est représenté par des relations «include» convergeant vers P1 - Authentification.

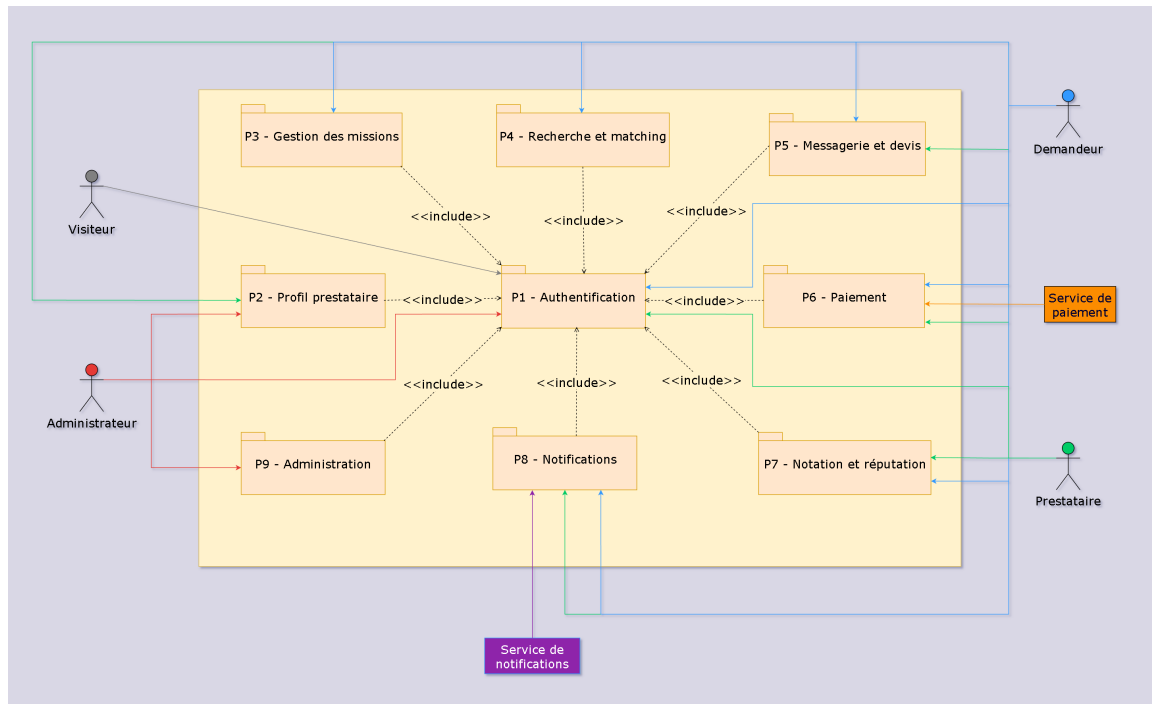


Figure 15 : Diagramme de packages de « SALISA »

2.2.1.5 Classification des cas d'utilisation (priorités, risques et itérations)

Le tableau 11 présente la classification des principaux cas d'utilisation selon leur priorité fonctionnelle, leur niveau de risque technique et l'itération de développement dans laquelle ils s'inscrivent. L'itération 1 regroupe les fonctionnalités essentielles au fonctionnement minimal du système. L'itération 2 contient les fonctionnalités importantes mais non bloquantes. L'itération 3 rassemble les fonctionnalités souhaitables à plus long terme, dont le développement dépend des ressources disponibles après la livraison des deux premières itérations.

UC	Libellé	Priorité	Risque	Itération
UC-01	s'inscrire	Élevée	Élevé	1
UC-02	valider le code OTP	Élevée	Élevé	1
UC-03	choisir son rôle	Élevée	Moyen	1
UC-06	compléter son profil	Élevée	Moyen	1
UC-11	publier une mission	Élevée	Moyen	1
UC-14	rechercher un prestataire	Élevée	Moyen	1
UC-19	envoyer un message	Élevée	Moyen	1
UC-20	envoyer un devis	Élevée	Moyen	1
UC-23	régler via MTN MoMo ou Airtel Money	Élevée	Élevé	1
UC-26	valider la mission	Élevée	Moyen	1
UC-28	laisser un avis	Élevée	Moyen	1
UC-30	recevoir des notifications	Élevée	Moyen	1
UC-32	tableau de bord admin	Élevée	Moyen	1
UC-12	utiliser l'assistant IA	Moyenne	Élevé	2
UC-16	score de matching	Moyenne	Élevé	2
UC-17	suggestions proactives	Moyenne	Moyen	2
UC-27	demander un retrait	Basse	Moyen	3

Tableau 11 : Classification des principaux cas d'utilisation de « SALISA »

2.2.2 Spécification détaillée

2.2.2.1 Description textuelle des cas d'utilisation (au plus 2 cu)

a- Description de UC-01 : s'inscrire via numéro de téléphone

Cas d'utilisation	UC-01 : s'inscrire via numéro de téléphone
Objectif	détaille les étapes permettant à un Visiteur de s'inscrire afin de devenir Utilisateur
Acteur principal	Visiteur
Acteurs secondaires	Système SALISA, Service de notifications
Date	le 23/04/2026
Responsables	Christophe MASSAMBA & Deo BATA
Version	1.0
Précondition	le visiteur n'a pas encore de compte sur SALISA
Déclencheur	le visiteur clique sur le bouton « S'inscrire »
Scénario nominal	1. Le visiteur saisit son numéro de téléphone au format +242 XXXXXXXXXX. 2. Le système valide le format et envoie un code OTP par SMS (UC-02). 3. Le visiteur saisit le code OTP reçu. 4. Le système vérifie le code (durée de validité : 5 minutes). 5. Le visiteur choisit son rôle : prestataire ou demandeur (UC-03). 6. Le compte est créé et le visiteur devient un utilisateur connecté.
Scénarios alternatifs	• Code OTP expiré : le visiteur peut en demander un nouveau. • 3 tentatives échouées : blocage temporaire de 10 minutes. • Numéro déjà existant : redirection vers la page de connexion.
Postcondition	le compte est créé ; le visiteur est devenu un utilisateur et peut accéder à toutes les fonctionnalités selon son rôle

Tableau 12 : Description textuelle de UC-01 : s'inscrire via numéro de téléphone

b- Description de UC-11 : publier une mission

Cas d'utilisation	UC-11 : publier une mission
Objectif	détaille les étapes permettant à un Demandeur de publier une mission
Acteur principal	Demandeur
Acteurs secondaires	Système SALISA, Service de notifications
Date	le 23/04/2026
Responsables	Christophe MASSAMBA & Deo BATA
Version	1.0
Précondition	l'utilisateur est connecté en tant que demandeur
Déclencheur	le demandeur clique sur « Publier une mission »
Scénario nominal	1. Le demandeur saisit le titre de la mission. 2. Il renseigne la description de son besoin (ou utilise l'assistant IA - UC-12). 3. Il sélectionne la catégorie et l'arrondissement. 4. Il indique le budget et le délai souhaité. 5. Il confirme et publie la mission. 6. Le système lance l'algorithme de matching et notifie les prestataires correspondants.
Scénarios alternatifs	• Le demandeur n'est pas connecté : redirection vers la page de connexion. • Aucun prestataire ne correspond : message « Aucun résultat proche ».
Postcondition	la mission est publiée et les prestataires correspondants sont notifiés

Tableau 13 : Description textuelle de UC-11 : publier une mission

2.2.2.2 Diagramme de séquence (au plus 2)

La figure 16 illustre le diagramme de séquence du cas d'utilisation UC-01 (s'inscrire via numéro de téléphone).

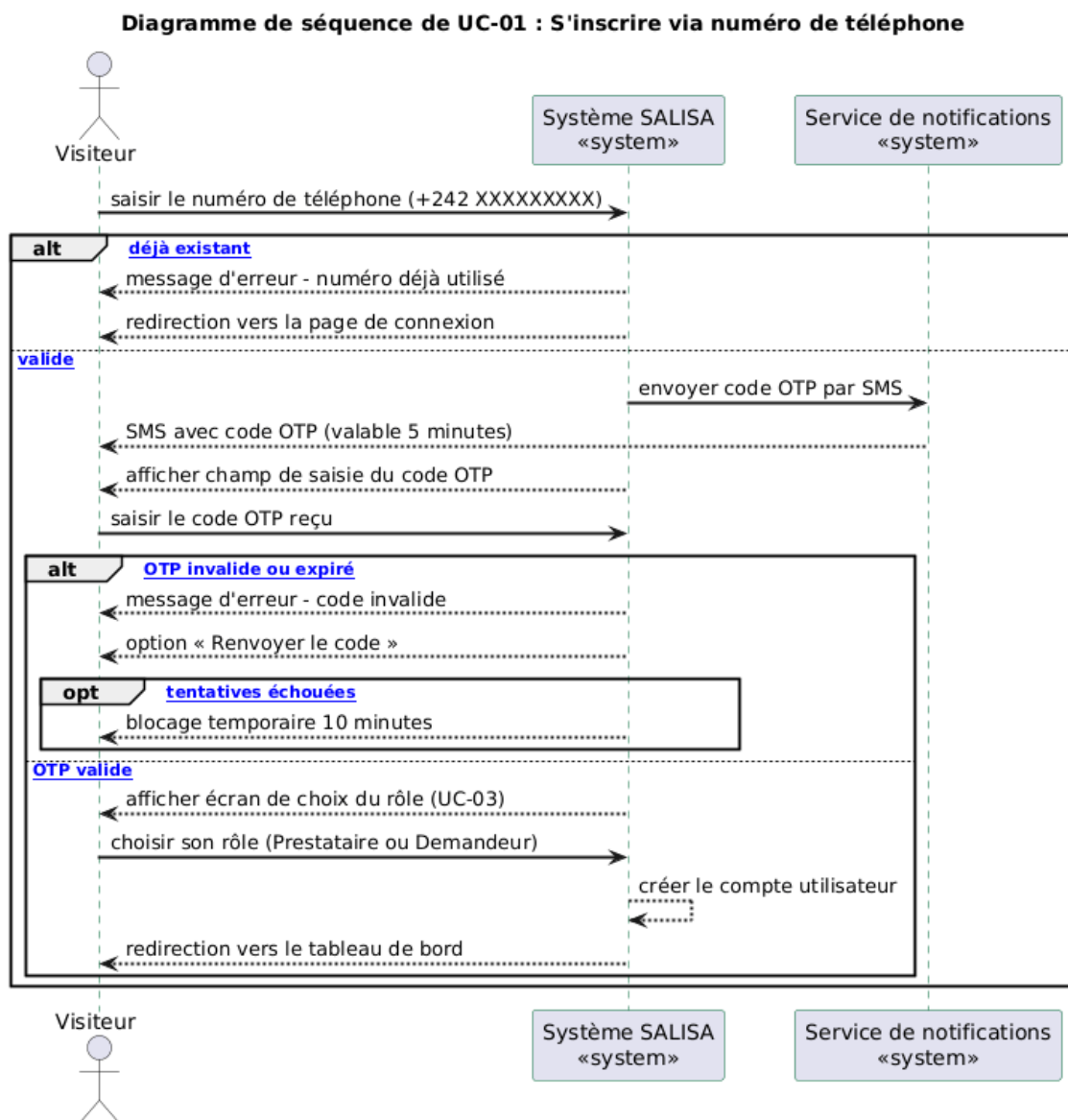


Figure 16 : Diagramme de séquence - inscription via numéro de téléphone (UC-01)

La figure 17 illustre le diagramme de séquence du processus de paiement d'une mission (UC-20, UC-21, UC-23 et UC-26). Ce scénario débute par l'envoi d'un devis par le prestataire, qui en constitue le véritable déclencheur.

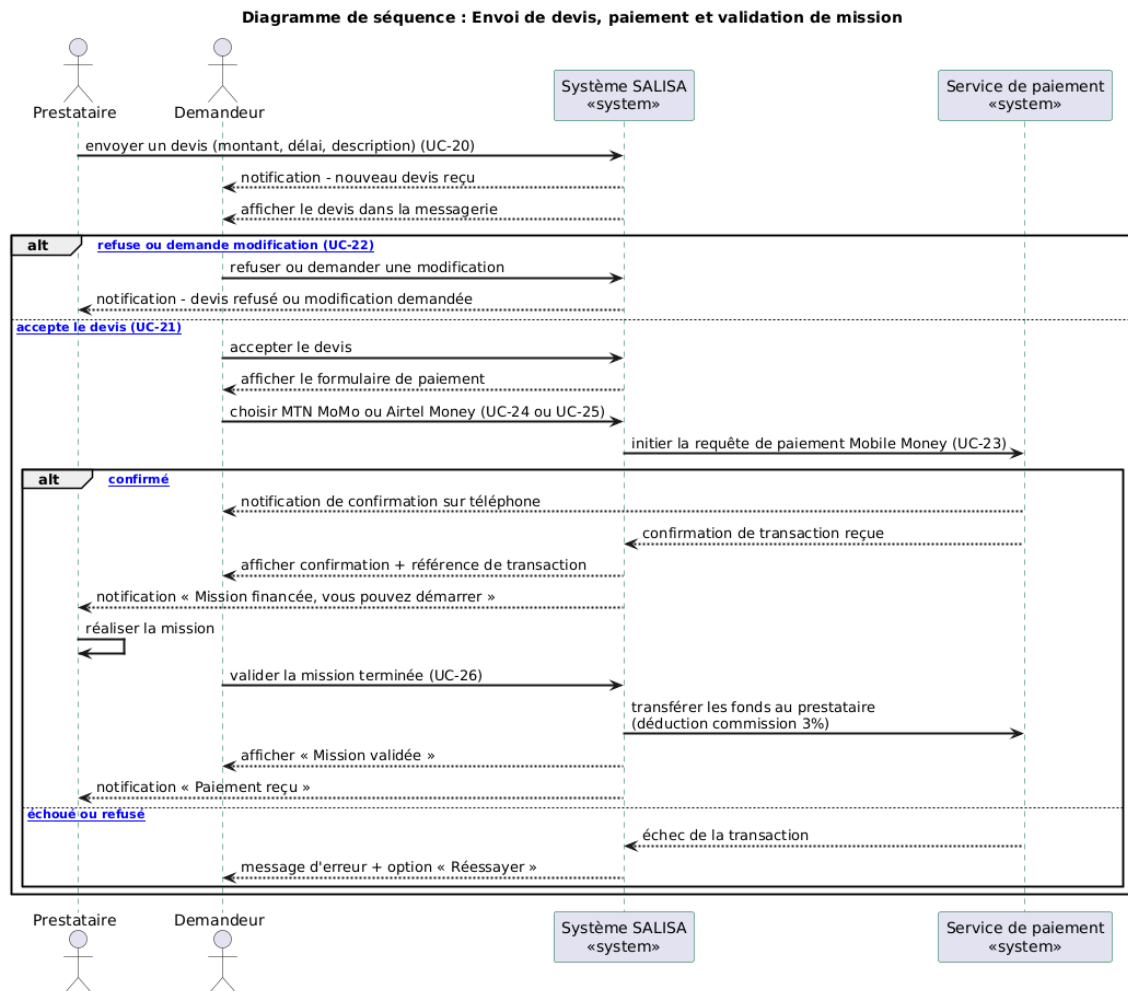


Figure 17 : Diagramme de séquence - envoi de devis, paiement et validation de mission

2.2.2.3 Diagramme d'activités (au plus 2)

La figure 18 présente le diagramme d'activités du processus de publication d'une mission et de sélection d'un prestataire, impliquant le Demandeur, le Système SALISA et le Prestataire.

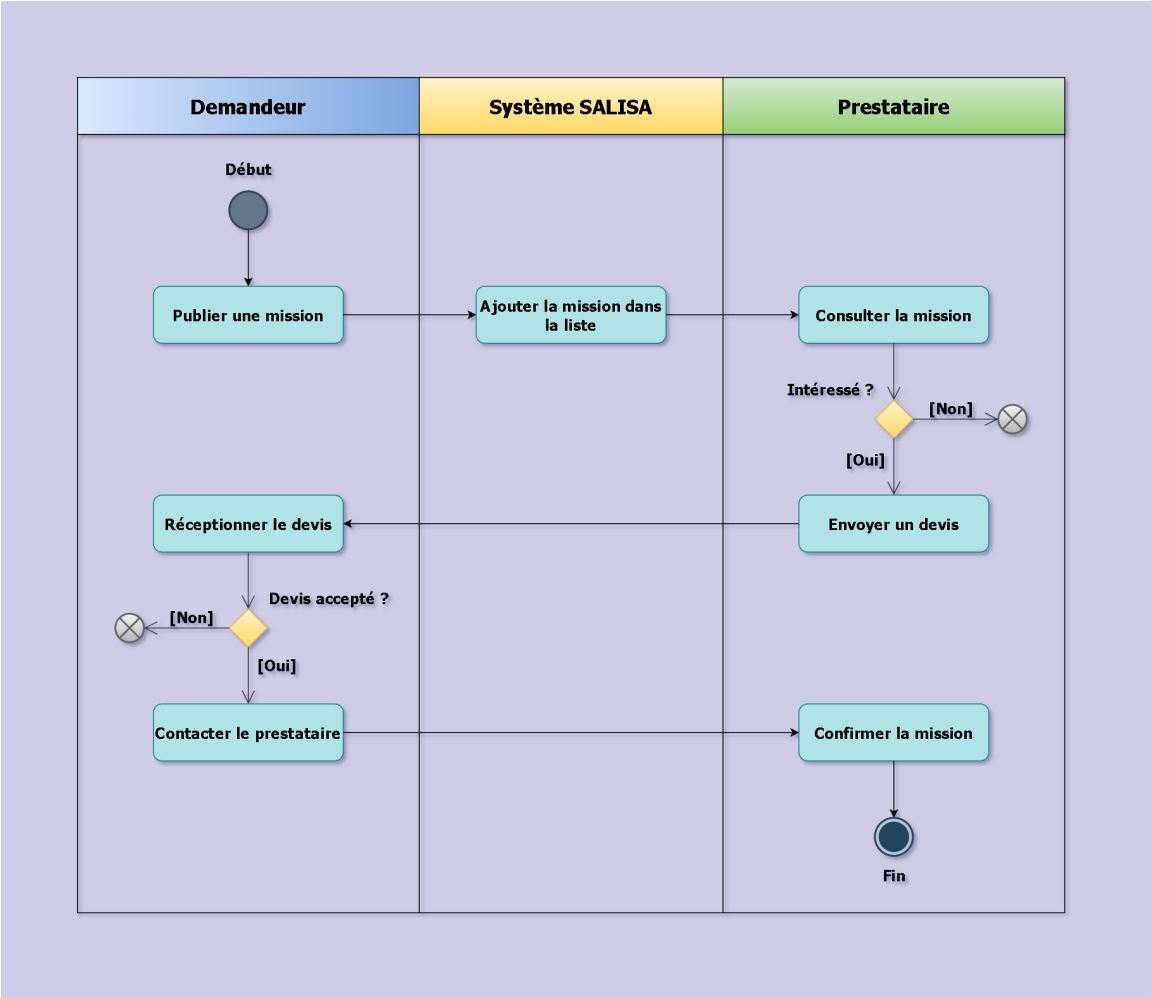


Figure 18 : Diagramme d'activités - publication de mission et sélection du prestataire

La figure 19 illustre le diagramme d'activités du processus de paiement mobile.

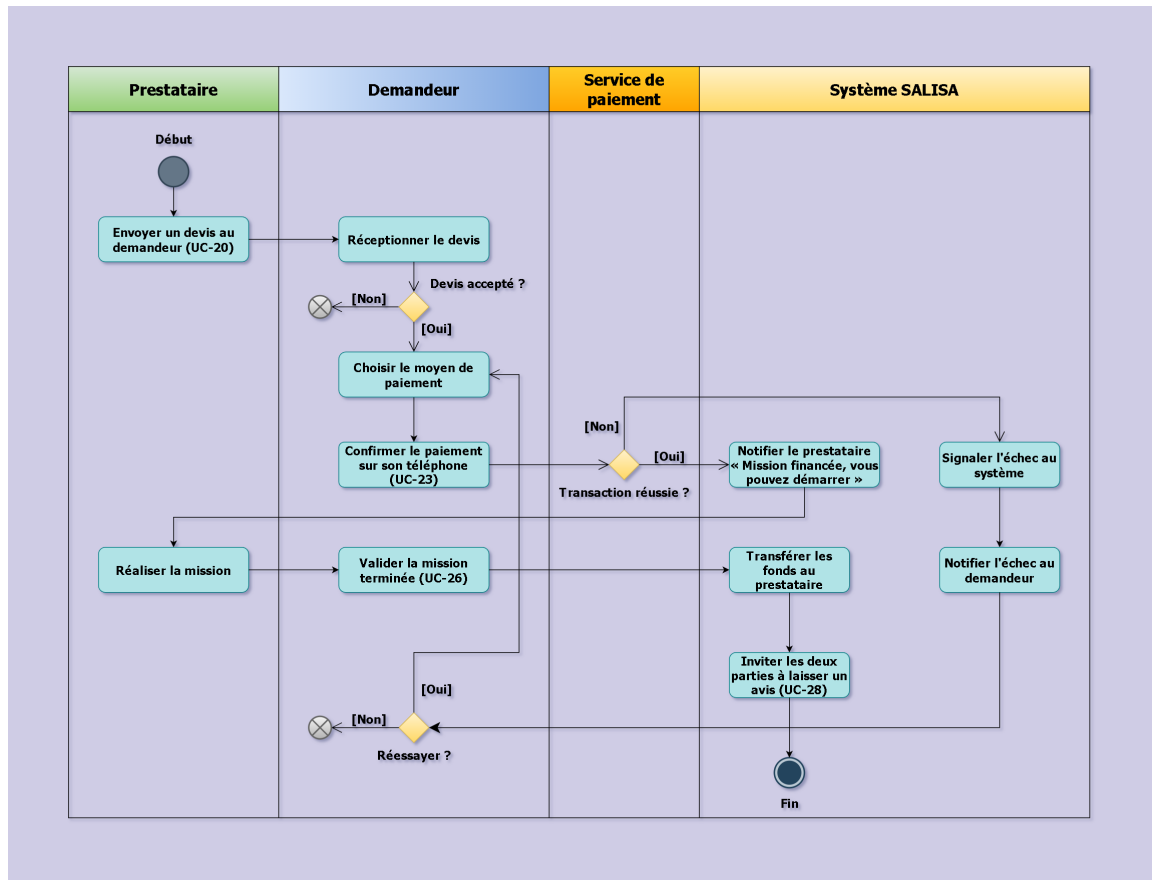


Figure 19 : Diagramme d'activités - processus de paiement mobile

2.2.3 Réalisation des cas d'utilisation

2.2.3.1 Quelques règles de gestion

Les règles de gestion définissent le comportement du système « SALISA » dans des situations précises [5]. Elles garantissent la cohérence des données et la fiabilité des transactions.

Gestion des comptes

- RG-001 : un numéro de téléphone ne peut être associé qu'à un seul compte ;
- RG-002 : un compte est associé à un seul rôle, soit Prestataire, soit Demandeur ;
- RG-003 : un compte est suspendu automatiquement après trois signalements validés par un administrateur ;
- RG-004 : lors de la suppression d'un compte, les données sont conservées 30 jours avant effacement définitif.

Gestion des missions et des prestataires

- RG-005 : un profil prestataire n'est visible dans la recherche qu'à partir de 70% de complétion ;
- RG-006 : un prestataire avec un compte gratuit ne peut postuler qu'à 3 missions par mois ;
- RG-007 : une mission sans activité depuis 30 jours est archivée automatiquement ;
- RG-008 : au maximum 10 prestataires sont notifiés par mission (les 10 meilleurs scores de matching).

Gestion des paiements

- RG-009 : la commission de « SALISA » est de 3% du montant total de chaque transaction ;
- RG-010 : le montant minimum d'une transaction est de 2 000 FCFA ;
- RG-011 : le paiement est déclenché uniquement après acceptation du devis par le demandeur ;
- RG-012 : les fonds sont transférés au prestataire après validation de la mission par le demandeur.

Gestion des avis

- RG-013 : un avis ne peut être laissé qu'après une mission avec transaction validée ;
- RG-014 : le délai pour laisser un avis est de 14 jours après la validation de la mission ;
- RG-015 : les avis sont définitifs et ne peuvent jamais être supprimés par le prestataire ou le demandeur.

2.2.3.2 Modèle du domaine (Diagramme de classe du système)

La figure 20 présente le diagramme de classes de « SALISA », qui représente les entités principales du système, leurs attributs, leurs méthodes et les relations qui les unissent [1].

Conception et réalisation d'une application cross-plateforme de mise en relation entre prestataires et clients pour services professionnels à Akieni

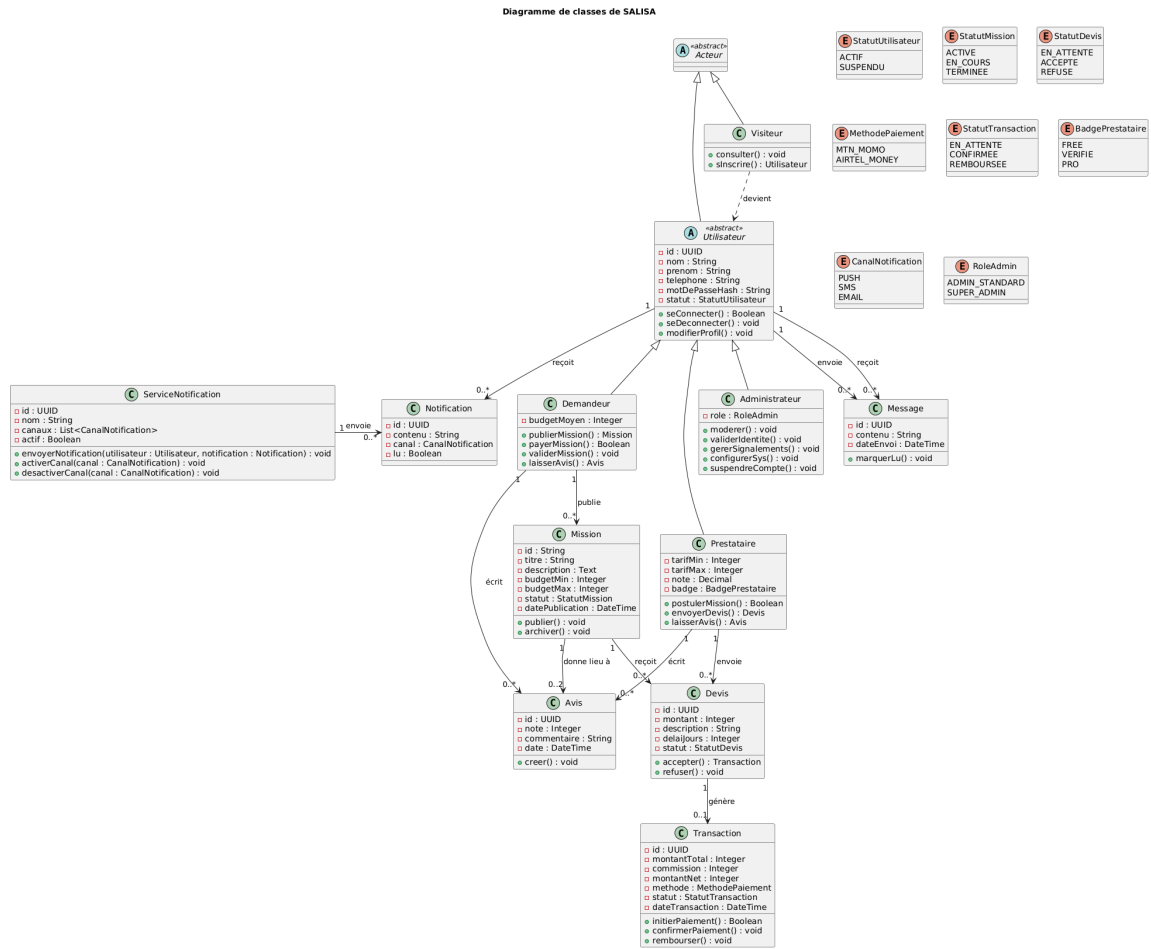


Figure 20 : Diagramme de classes du système « SALISA »

2.2.3.3 Diagramme d'Objets

La figure 21 illustre un exemple d'instanciation du système « SALISA », correspondant au scénario où un demandeur publie une mission, un prestataire y répond avec un devis, et le paiement est effectué via MTN MoMo.

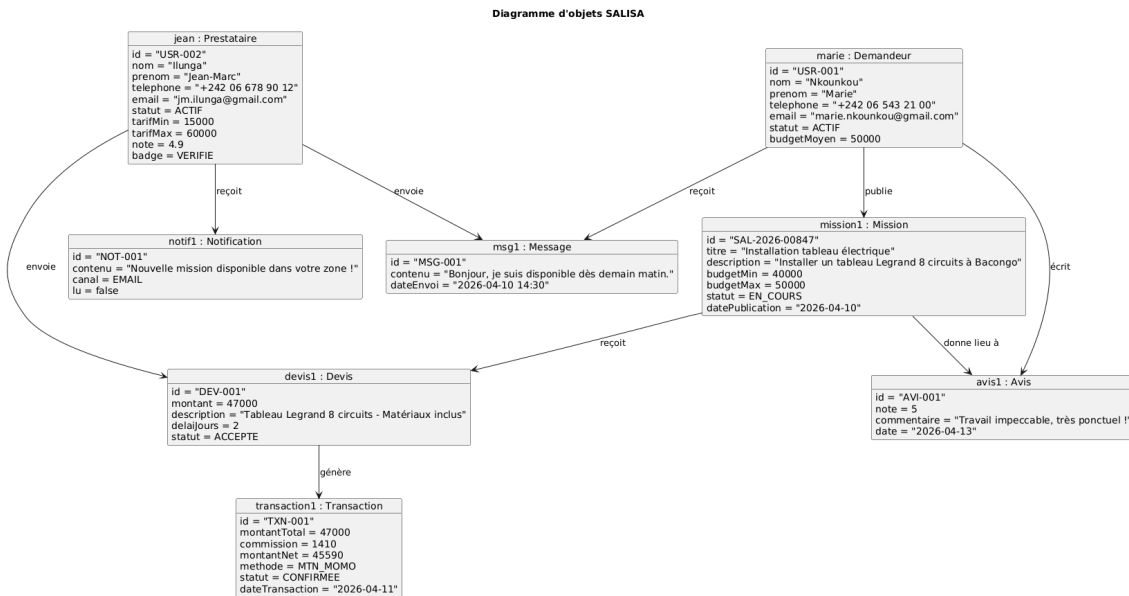


Figure 21 : Diagramme d'objets - scénario de publication et réponse à une mission

2.2.4 Conception architecturale

2.2.4.1 Architecture logicielle

« SALISA » repose sur une architecture modulaire organisée autour de cinq couches principales.

Le **front-end** est développé en Next.js 14 [7], un framework React qui permet de générer des pages côté serveur (SSR) et des pages statiques (SSG). Il assure l'aspect PWA mobile-first et l'optimisation pour les réseaux lents.

Le **back-end** est développé en Node.js avec Express.js. Il expose une API REST pour toutes les opérations classiques et utilise Socket.io pour la messagerie en temps réel via WebSocket.

La **base de données** principale est PostgreSQL [6], qui stocke toutes les données persistantes (utilisateurs, missions, transactions, avis). Redis est utilisé comme cache pour les sessions et les résultats de recherche fréquents.

Le **Service de matching** est un microservice Python développé avec FastAPI. Il calcule les scores de compatibilité entre les missions et les prestataires sur la base de huit critères pondérés : localisation, disponibilité, compétences, budget, note, fiabilité, temps de réponse et niveau de badge.

Le **Service de notifications** est un worker asynchrone qui envoie les alertes via trois canaux : les notifications push PWA, les e-mails et les SMS.

2.2.4.2 Architecture globale de la solution (Diagramme de déploiement)

La figure 22 présente le diagramme de déploiement de « SALISA », qui montre la répartition des composants sur les équipements matériels et les services en ligne.

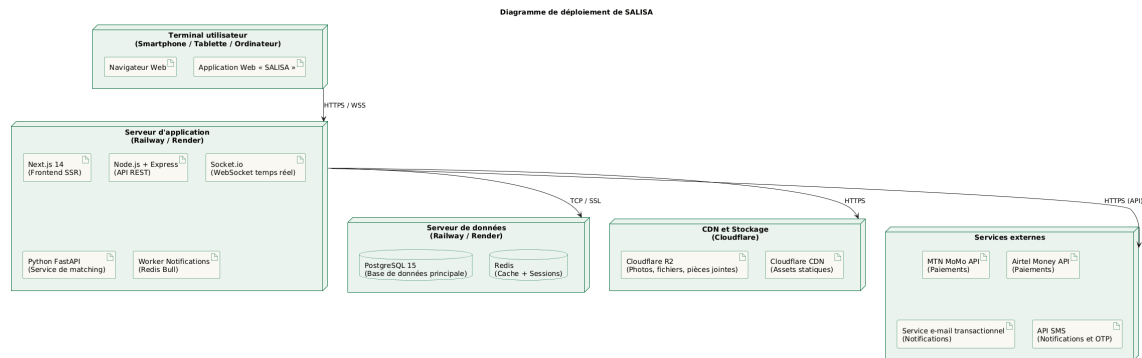


Figure 22 : Diagramme de déploiement de « SALISA »

Troisième Partie

ÉVALUATION ET RÉALISATION DU PROJET

Chapitre I : L'évaluation du projet

1.1 Organisation du projet

« SALISA » a été réalisé dans le cadre de l'obtention du diplôme de Licence Professionnelle en Génie Logiciel à l'École Supérieure de Gestion et d'Administration des Entreprises (ESGAE) de Brazzaville. Il s'agit d'un projet de fin d'études mené en binôme, sous la supervision d'un Directeur de Projet (DP).

La conduite du projet repose sur des séances de travail hebdomadaires avec le Directeur de Projet, tenues le plus souvent les samedis à l'école, ou en ligne via Meet les jours ordinaires. Ces séances permettent de faire le point sur l'avancement, de valider les livrables et d'orienter les prochaines étapes. La méthode de développement adoptée est le **2TUP**, couplée au langage de modélisation **UML**.

1.2 Intervenants

Plusieurs acteurs ont participé à la réalisation de « SALISA ». Le tableau 14 les présente avec leur fonction et leur rôle dans le projet.

Intervenant	Fonction	Titre / Rôle
M. Christophe Darly MASSAMBA BOUESSO	Étudiant en LPGL	Concepteur et développeur
M. Grâce Deo BATA	Étudiant en LPGL	Concepteur et développeur
M. Ornel Djenifer KIGOMA	Ingénieur informaticien	Directeur de projet, enseignant à l'ESGAE

Tableau 14 : Intervenants du projet « SALISA »

1.3 Planification des tâches

La réalisation de « SALISA » a été découpée en plusieurs phases successives et parfois parallèles, couvrant la période allant du 5 janvier 2026, date d'affichage officiel des binômes et de leurs thèmes, jusqu'à la soutenance prévue le samedi 20 juin 2026. Le tableau 15 présente l'ensemble des tâches réalisées avec leurs dates de début et de fin.

N°	Tâche	Début	Fin
1	Cadrage du projet et du thème	05/01/2026	14/02/2026
2	Prise de contact et collecte des informations sur la structure d'accueil	05/01/2026	14/02/2026
3	Étude de l'existant et analyse des besoins	15/02/2026	28/02/2026
4	Rédaction du rapport de soutenance	14/02/2026	29/05/2026
5	Modélisation UML - diagrammes de cas d'utilisation	01/03/2026	31/03/2026
6	Maquette et prototype de l'interface (UI/UX)	01/03/2026	25/03/2026
7	Modélisation UML - diagrammes de séquence et d'activités	01/04/2026	18/04/2026
8	Modélisation UML - diagrammes de classes et d'objets	01/04/2026	18/04/2026
9	Modélisation de la base de données	19/04/2026	28/04/2026
10	Développement back-end (API REST, WebSocket, authentification)	16/04/2026	18/05/2026
11	Développement front-end (interfaces utilisateur)	05/05/2026	05/06/2026
12	Intégration des API de paiement mobile (MTN MoMo, Airtel Money)	10/05/2026	05/06/2026
13	Intégration et déploiement sur serveur VPS	06/06/2026	10/06/2026
14	Tests et corrections	20/05/2026	12/06/2026
15	Révision finale et dépôt du document	30/05/2026	01/06/2026
16	Préparation de la présentation	02/06/2026	15/06/2026
17	Préparation et répétitions de soutenance	16/06/2026	19/06/2026
18	Soutenance	20/06/2026	20/06/2026

Tableau 15 : Planification des tâches du projet « SALISA »

Pour compléter cette vue d'ensemble, le tableau 23 présente la planification telle qu'elle a été saisie et suivie dans MS Project. Il détaille, pour chaque tâche, la durée calculée, les dates de

Conception et réalisation d'une application cross-plateforme de mise en relation entre prestataires et clients pour services professionnels à Akieni

début et de fin, ainsi que les dépendances entre les tâches (prédécesseurs). Cette vue permet notamment de visualiser la structure hiérarchique du projet, avec ses phases récapitulatives et ses tâches élémentaires.

	i	Mode Tâche	Nom de la tâche	Durée	Début	Fin	Prédécesseurs
1			Conception et réalisation d'une application cross-plateforme de mise en relation entre prestataires et clients pour services professionnels à Akieni	167 jours?	05/01/2026	20/06/2026	
2			Prérequis et cadrage	41 jours?	05/01/2026	14/02/2026	
3			Cadrage du projet et du thème	41 jours?	05/01/2026	14/02/2026	
4			Prise de contact sur la structure d'accueil	41 jours	05/01/2026	14/02/2026	
5			Documentation et analyse	105 jours	14/02/2026	29/05/2026	
6			Étude de l'existant et analyse des besoins	14 jours	15/02/2026	28/02/2026	4
7			Rédaction du rapport de soutenance	105 jours	14/02/2026	29/05/2026	
8			Modélisation UML	59 jours?	01/03/2026	28/04/2026	
9			Diagrammes de cas d'utilisation	31 jours?	01/03/2026	31/03/2026	6
10			Maquette et prototype de l'interface	25 jours?	01/03/2026	25/03/2026	6
11			Diagrammes de séquence et d'activités	18 jours	01/04/2026	18/04/2026	9
12			Diagrammes de classes et d'objets	18 jours	01/04/2026	18/04/2026	9
13			Modélisation de la base de données	10 jours	19/04/2026	28/04/2026	12
14			Développement de l'application	58 jours	16/04/2026	12/06/2026	
15			Développement back-end	33 jours	16/04/2026	18/05/2026	9
16			Développement front-end	32 jours	05/05/2026	05/06/2026	10
17			Intégration des API de paiement mobile	27 jours	10/05/2026	05/06/2026	
18			Intégration et déploiement sur VPS	5 jours	06/06/2026	10/06/2026	16;17
19			Tests et corrections	24 jours	20/05/2026	12/06/2026	
20			Finalisation	22 jours?	30/05/2026	20/06/2026	
21			Révision finale et dépôt du document	3 jours	30/05/2026	01/06/2026	7
22			Préparation de la présentation	14 jours	02/06/2026	15/06/2026	21
23			Préparation et répétitions de soutenance	4 jours	16/06/2026	19/06/2026	22
24			Soutenance	1 jour?	20/06/2026	20/06/2026	23

Figure 23 : Liste détaillée des tâches du projet « SALISA » sous MS Project

1.4 Diagramme de Gantt

Le diagramme de Gantt, réalisé avec MS Project, offre une représentation visuelle du déroulement chronologique du projet. Il met en évidence les chevauchements entre les tâches menées en parallèle (en l'occurrence la rédaction du document et le développement de l'application), ainsi que les dépendances entre certaines phases. La figure 24 illustre ce planning.

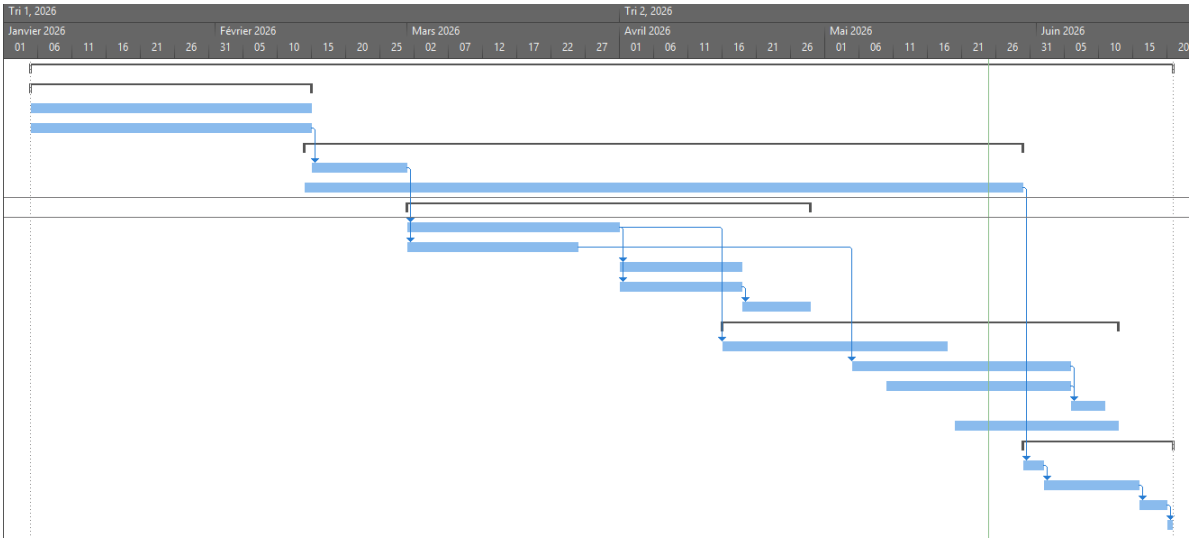


Figure 24 : Diagramme de Gantt du projet « SALISA »

1.5 Estimation des charges

L’estimation des charges regroupe l’ensemble des ressources nécessaires à la réalisation de « SALISA ». Elle prend en compte les ressources humaines, les services en ligne, les logiciels et les équipements matériels. Les montants sont exprimés en Francs CFA.

**Conception et réalisation d'une application cross-plateforme de mise en relation entre prestataires
et clients pour services professionnels à Akieni**

Ressources	Quantité / Mois	Prix unitaire (en FCFA)	Prix total (en FCFA)
Ressources humaines			
2 Développeurs	5	300 000	3 000 000
Services en ligne			
Connexion internet	12	25 000	300 000
Serveur VPS (hébergement)	12	15 000	180 000
Ressources logicielles			
LaTeX / TeXstudio	1	Gratuit	Gratuit
VS Code	1	Gratuit	Gratuit
PostgreSQL	1	Gratuit	Gratuit
Node.js / Redis	1	Gratuit	Gratuit
Git et GitHub	1	Gratuit	Gratuit
Docker	1	Gratuit	Gratuit
Draw.io Desktop	1	Gratuit	Gratuit
Postman (tests API)	1	Gratuit	Gratuit
PlantUML	1	Gratuit	Gratuit
MS Project (abonnement annuel Plan 1)	12	6 250	75 000
Ressources matérielles			
Ordinateur portable	2	650 000	1 300 000
Montant total			4 855 000

Tableau 16 : Estimation des charges du projet « SALISA »

Chapitre II : Les outils et les techniques utilisés

Pour concevoir et réaliser « SALISA », nous avons mobilisé un ensemble de techniques et d'outils modernes, choisis pour leur adéquation avec les besoins du projet et le contexte de développement. Ce chapitre présente les approches techniques retenues ainsi que les outils concrets utilisés tout au long du projet.

2.1 Présentation des techniques

Architecture REST

L'architecture REST (*Representational State Transfer*) est un style de conception d'API qui repose sur le protocole HTTP. Dans « SALISA », le back-end expose une API REST permettant aux différentes parties de l'application de communiquer de façon structurée : le front-end envoie des requêtes HTTP (GET, POST, PUT, DELETE) et reçoit des réponses au format JSON. Cette architecture facilite la séparation claire entre l'interface utilisateur et la logique métier, et rend le système plus facile à faire évoluer.

WebSocket

Le WebSocket est un protocole de communication qui permet d'établir une connexion permanente et bidirectionnelle entre le navigateur de l'utilisateur et le serveur. Contrairement à HTTP qui fonctionne en mode requête-réponse, le WebSocket permet au serveur d'envoyer des données au client en temps réel, sans que ce dernier n'ait besoin de les demander. Dans « SALISA », cette technologie est utilisée pour la messagerie instantanée entre demandeurs et prestataires, grâce à la bibliothèque Socket.io.

OTP (One-Time Password)

L'OTP est un code à usage unique, valable pendant une courte durée, envoyé à l'utilisateur par SMS lors de son inscription ou de sa connexion. Il remplace le mot de passe classique et renforce la sécurité en s'assurant que la personne qui s'inscrit possède bien le numéro de téléphone qu'elle a renseigné. Dans « SALISA », ce mécanisme est utilisé pour valider les comptes lors de l'inscription (UC-01 et UC-02).

JWT (JSON Web Token)

Le JWT est un format de jeton d'authentification sécurisé. Une fois connecté, l'utilisateur reçoit un JWT que son navigateur joint à chaque requête envoyée au serveur. Cela permet au

serveur de vérifier l'identité de l'utilisateur sans avoir à consulter la base de données à chaque appel. Dans « SALISA », le JWT est utilisé pour sécuriser toutes les routes de l'API qui nécessitent d'être authentifié.

Paiement mobile via API

Le paiement mobile repose sur l'intégration des API officielles de MTN MoMo et d'Airtel Money. Lorsqu'un demandeur confirme un paiement, « SALISA » envoie une requête à l'API de l'opérateur concerné, qui déclenche une notification sur le téléphone du demandeur pour qu'il confirme la transaction. Une fois la confirmation reçue, les fonds sont transférés et le prestataire est notifié que la mission est financée.

Matching par score

Le matching est le mécanisme qui permet à « SALISA » de proposer automatiquement les prestataires les plus adaptés à une mission publiée. Il s'appuie sur un microservice Python développé avec FastAPI, qui calcule pour chaque prestataire un score de compatibilité basé sur huit critères pondérés : la localisation, la disponibilité, les compétences, le budget, la note moyenne, le taux de fiabilité, le temps de réponse habituel et le niveau de badge. Les dix prestataires avec les meilleurs scores sont ensuite notifiés.

2.2 Présentation des outils

Next.js 14

Next.js est un framework JavaScript basé sur React, développé par Vercel. Il permet de créer des applications web avec rendu côté serveur (SSR) et génération de pages statiques (SSG), ce qui améliore les performances et le référencement. Dans « SALISA », Next.js constitue la couche front-end de l'application et prend en charge l'interface utilisateur accessible depuis tous les types de terminaux.

Node.js et Express.js

Node.js est un environnement d'exécution JavaScript côté serveur, reconnu pour sa légèreté et ses performances sur les applications à forte concurrence. Express.js est un framework minimaliste qui simplifie la création d'API REST avec Node.js. Dans « SALISA », ces deux outils forment le socle du back-end et gèrent l'ensemble des requêtes provenant du front-end.

PostgreSQL

PostgreSQL est un système de gestion de bases de données relationnelles open source, reconnu pour sa robustesse et sa conformité aux standards SQL. Il stocke dans « SALISA » toutes les données persistantes du système, telles que les comptes utilisateurs, les missions, les devis, les transactions et les avis.

Redis

Redis est une base de données en mémoire utilisée principalement comme système de cache. Dans « SALISA », il est utilisé pour stocker les sessions utilisateurs et les résultats de recherche fréquents, ce qui réduit la charge sur PostgreSQL et accélère les temps de réponse de l'application.

Python et FastAPI

Python est un langage de programmation polyvalent, particulièrement apprécié pour le traitement de données et l'intelligence artificielle. FastAPI est un framework Python moderne qui permet de créer des API performantes très rapidement. Dans « SALISA », Python et FastAPI constituent le microservice de matching, chargé de calculer les scores de compatibilité entre missions et prestataires.

Visual Studio Code

Visual Studio Code (VS Code) est un éditeur de code gratuit, développé par Microsoft. Il est utilisé par les deux membres du binôme pour la rédaction du code de l'application, grâce à ses nombreuses extensions qui facilitent le développement JavaScript, Python et la gestion de projets.

Git et GitHub

Git est un outil de gestion de versions qui permet de suivre l'historique des modifications apportées au code source. GitHub est la plateforme en ligne qui héberge le dépôt Git du projet. Ensemble, ils permettent au binôme de travailler en parallèle sur le code, de fusionner les contributions et de conserver une trace de chaque évolution du projet.

Postman

Postman est un outil qui permet de tester les API REST de façon simple et visuelle. Il a été utilisé tout au long du développement de « SALISA » pour vérifier que chaque route de l'API

répond correctement avant d'être connectée au front-end.

Docker

Docker est un outil de conteneurisation qui permet d'empaqueter une application et toutes ses dépendances dans un conteneur portable. Il garantit que l'application fonctionne de la même façon sur tous les environnements, que ce soit en développement local ou en production sur le serveur VPS.

Draw.io Desktop

Draw.io est un logiciel de création de diagrammes, disponible en version bureau. Il a été utilisé pour réaliser certains diagrammes UML du projet, notamment le diagramme de contexte statique et le diagramme de packages, qui nécessitaient un contrôle précis de la disposition visuelle des éléments.

PlantUML

PlantUML est un outil open source qui permet de générer des diagrammes UML à partir de scripts texte. Il a été utilisé pour produire la majorité des diagrammes de modélisation du projet, tels que les diagrammes de cas d'utilisation, de séquence, d'activités, de classes et d'objets.

LaTeX et TeXstudio

LaTeX est un système de composition de documents qui permet de produire des documents de haute qualité typographique. TeXstudio est l'éditeur utilisé pour rédiger et compiler ce rapport. Le choix de LaTeX a permis de gérer automatiquement la numérotation des figures et des tableaux, les références croisées et la mise en page générale du document.

Microsoft Project

Microsoft Project (MS Project) est un logiciel de gestion de projet édité par Microsoft. Il a été utilisé pour planifier les tâches du projet, définir leurs dépendances et générer le diagramme de Gantt présenté au point 1.4 de ce chapitre.

2.3 Les captures d'écran

[À insérer : captures d'écran de l'application SALISA illustrant les principales fonctionnalités réalisées]

Conclusion

Au terme de ce travail, nous avons conçu et réalisé « SALISA », une application cross-plateforme de mise en relation entre prestataires et clients pour services professionnels, développée pour le compte d'Akieni.

Ce travail nous a permis de parcourir l'ensemble du cycle de développement logiciel, depuis l'analyse des besoins jusqu'à la réalisation d'une solution fonctionnelle, en passant par la conception architecturale et la modélisation UML selon la démarche **2TUP**. Nous avons pu mesurer concrètement la complémentarité entre un langage de modélisation expressif comme UML et un processus de développement structuré comme le 2TUP, dont l'approche en deux branches parallèles s'est révélée particulièrement adaptée à la complexité de « SALISA ».

Sur le plan fonctionnel, « SALISA » propose un ensemble de fonctionnalités répondant aux besoins réels du marché congolais : mise en correspondance intelligente entre prestataires et demandeurs, système de confiance à plusieurs niveaux avec badges de certification, paiement sécurisé via MTN MoMo et Airtel Money, messagerie en temps réel et notifications multi-canaux par e-mail et SMS. L'application est conçue comme une *Progressive Web Application* (PWA), optimisée pour les réseaux mobiles et accessible sans installation.

Ce travail illustre la capacité du numérique à structurer des marchés informels et à créer de la valeur économique locale, tout en répondant aux contraintes spécifiques du contexte africain. Il constitue une contribution concrète à la transformation numérique en cours en République du Congo.

En perspective, plusieurs axes d'amélioration pourront être envisagés : le déploiement d'un module de machine learning pour affiner l'algorithme de mise en correspondance, l'extension de « SALISA » vers d'autres villes du pays comme Pointe-Noire, le développement d'une application mobile native, et la mise en place d'un programme d'ambassadeurs pour accélérer son adoption.

Ce travail, mené dans le cadre de notre formation en Licence Professionnelle Génie Logiciel à l'ESGAE, nous a permis de consolider nos compétences techniques et méthodologiques, tout en relevant un défi concret ancré dans les réalités de notre pays.

Ko salisa biso nyonso - Nous nous entraïdons tous.

SECTION III

Table des matières

SECTION I

Dédicace	I
Remerciements	II
Liste des figures	III
Liste des tableaux	IV
Abréviations et Sigles	V
Sommaire	VI

SECTION II

Introduction	1
Première Partie : Présentation Générale	2
Chapitre I La structure d’accueil et le sujet	3
1.1 La structure d’accueil	3
1.1.1 Historique	3
1.1.2 Missions	3
1.1.3 Organigramme Général	4
1.1.4 Attributions des structures	4
1.1.5 Situation informatique	5
1.1.5.1 Personnel informatique	5
1.1.5.2 Matériels informatiques	5
1.1.5.3 Logiciels informatiques	5
1.2 Le sujet	6
1.2.1 Contexte du sujet	6
1.2.2 Problématique du sujet	6
1.2.3 Intérêts du sujet	6
1.3 Concepts liés au sujet	7

Deuxième Partie : Analyse et Conception	9
Chapitre I Étude préliminaire et Méthode	10
1.1 Étude de l'existant	10
1.1.1 Description des activités (situation actuelle)	10
1.1.2 Critique de l'existant (limites)	10
1.1.3 Proposition des solutions	11
1.1.4 Choix de la solution	13
1.2 Méthode	13
1.2.1 Le langage de modélisation	14
1.2.1.1 Présentation d'UML	14
1.2.1.2 Les types de diagrammes UML	15
1.2.2 Le processus de développement	16
1.2.2.1 Présentation du Processus Unifié (UP)	16
1.2.2.2 Présentation du processus 2TUP	16
1.2.2.3 2TUP et UML	17
Chapitre II Analyse et Conception	19
2.1 Analyse du besoin	19
2.1.1 Besoins Fonctionnels	19
2.1.2 Besoins Techniques	20
2.2 Conception du système	20
2.2.1 Spécifications fonctionnelles	20
2.2.1.1 Identification des acteurs	20
2.2.1.2 Diagramme de contexte statique	21
2.2.1.3 Identification des cas d'utilisation	22
2.2.1.4 Relation entre les cas d'utilisation	25
2.2.1.5 Classification des cas d'utilisation (priorités, risques et itérations)	31
2.2.2 Spécification détaillée	32
2.2.2.1 Description textuelle des cas d'utilisation (au plus 2 cu)	32
2.2.2.2 Diagramme de séquence (au plus 2)	35
2.2.2.3 Diagramme d'activités (au plus 2)	36
2.2.3 Réalisation des cas d'utilisation	38
2.2.3.1 Quelques règles de gestion	38
2.2.3.2 Modèle du domaine (Diagramme de classe du système)	39

2.2.3.3	Diagramme d’Objets	40
2.2.4	Conception architecturale	41
2.2.4.1	Architecture logicielle	41
2.2.4.2	Architecture globale de la solution (Diagramme de déploiement)	42
Troisième Partie : Évaluation et Réalisation du Projet		43
Chapitre I L’évaluation du projet		44
1.1	Organisation du projet	44
1.2	Intervenants	44
1.3	Planification des tâches	44
1.4	Diagramme de Gantt	46
1.5	Estimation des charges	47
Chapitre II Les outils et les techniques utilisés		49
2.1	Présentation des techniques	49
2.2	Présentation des outils	50
2.3	Les captures d’écran	52
Conclusion		53
 SECTION III		
Table des matières		a
Bibliographie		d
Annexes		e

Bibliographie

- [1] Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, Boston, 2 edition, 2005.
- [2] Martin Fowler. *UML Distilled : A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley, Boston, 3 edition, 2003.
- [3] Ivar Jacobson, Grady Booch, and James Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, Reading, Massachusetts, 1999.
- [4] MTN Group. Mtn mobile money developer documentation. <https://momodeveloper.mtn.com>, 2024. Consulté le 12 janvier 2026.
- [5] Pascal Roques and Franck Vallée. *UML en action : De l'analyse des besoins à la conception*. Eyrolles, Paris, 4 edition, 2007.
- [6] The PostgreSQL Global Development Group. PostgreSQL 15 documentation. <https://www.postgresql.org/docs/15/>, 2023. Consulté le 18 février 2026.
- [7] Vercel. Next.js documentation. <https://nextjs.org/docs>, 2024. Consulté le 5 février 2026.

Annexes

[À compléter : tout document complémentaire jugé utile - code source, captures d'écran supplémentaires, questionnaires, données de test, user stories du projet, etc.]